

WILP, BITS Pilani

MATHEMATICAL FOUNDATIONS FOR ML & DS · COMPANION READER

Systems of Linear Equations

Starting from zero: what a vector is, what a matrix is,
and how they let a computer learn from data

Session 1 · No prior linear algebra assumed · Every example worked out in full

Before you begin

This reader assumes you remember school algebra — what $2x + 3 = 11$ means and how to find x . Nothing more. Every new word is introduced with an everyday example first, a picture second, and the symbols last. You will meet five kinds of coloured box:

- **Blue — the big idea.** No symbols. Read only the blue boxes and you still get the story.
- **Amber — an everyday picture.** A shop, a recipe, a bill. Something you already understand, used as a handrail.
- **Green — a worked example.** Real numbers, every step shown. Cover the box and try it yourself first.
- **Red — watch out.** The mistake almost everybody makes here.
- **Purple — remember this.** The one sentence to carry forward.

There are also **navy boxes** answering “why does machine learning care about this?” — because that is the whole reason this subject is on your timetable.

The route we will take

Part I. The two objects. A vector is a list of numbers. A matrix is a grid of numbers. We build both from scratch, learn the only two things you may do to them, and see why a table of data is literally a matrix.

Part II. Clues about hidden numbers. What a linear equation is, why a canteen bill is one, and the surprising fact that a set of such clues can have exactly zero, exactly one, or endlessly many answers — but never two.

Part III. Matrix arithmetic. Adding, scaling, multiplying, flipping and undoing grids of numbers. This is the toolbox.

Part IV. $Ax = b$ and the one method that cracks it. Everything above packs into three symbols, and one tidy procedure — Gaussian elimination — solves any such system, every time.

• The whole session in one sentence

Data becomes a grid of numbers; a question about that data becomes a set of simple clues about unknown numbers; and there is one reliable procedure that either finds those unknowns or proves they cannot exist.

Part I · The two objects: vectors and matrices

Lists of numbers, grids of numbers, and the only two things you may do to them

1 One number is not enough

Suppose a friend asks you to describe a flat you saw yesterday. You could answer with a single number — “it costs 28”. But that leaves out almost everything. How big was it? How many bedrooms? How old is the building? A single number cannot carry all of that.

So you answer with a short list instead:

800 square feet, 2 bedrooms, 12 years old

Strip away the words and keep the numbers in a fixed order — size first, bedrooms second, age third — and you get $(8, 2, 12)$, with size counted in hundreds of square feet. That list is a **vector**. That is genuinely all a vector is.

▷ A vector is a list of numbers, in a fixed order

Nothing more mysterious than that. The two words that matter are *list* — several numbers travelling together as one object — and *fixed order* — slot one always means the same thing, slot two always means the same thing, and so on. Swap two numbers around and you have described something else entirely.

Vectors are already all around you

Every one of these is a vector, whether or not anybody calls it that:

- A day’s weather: $(31^\circ\text{C temperature, 78\% humidity, 4 mm rain})$.
- A student’s marks in three subjects: $(72, 65, 88)$.
- A colour on a screen: $(255, 140, 0)$ — red, green and blue amounts. That one is orange.
- Your shopping for the week: $(2 \text{ kg rice, 1 kg dal, 3 litres milk})$.
- A location: $(28.61 \text{ latitude, } 77.21 \text{ longitude})$.

In every case the order carries meaning. $(72, 65, 88)$ and $(88, 65, 72)$ are different report cards for different students.

Writing a vector down

Mathematicians write a vector standing up, in a column, wrapped in tall brackets, and give it a bold name:

$$\mathbf{v} = \begin{bmatrix} 8 \\ 2 \\ 12 \end{bmatrix}.$$

The individual numbers are the **components** or **entries**. Writing it flat as $(8, 2, 12)$ means exactly the same thing and saves paper — this reader uses both. A list of n real numbers is said to live in $\mathbb{R}^n$, read aloud as “R-n”. Our flat lives in $\mathbb{R}^3$ because it has three numbers in it. The $\mathbb{R}$ is just a reminder that the entries are ordinary real numbers, and the n tells you how many slots there are.

Three small facts, worth fixing now

One — the dimension is the number of slots. $(8, 2, 12)$ is a three-dimensional vector because it has three numbers in it. Nothing to do with how big those numbers are: $(1000, 2)$ is only two-dimensional, and $(1, 1, 1, 1, 1, 1)$ is six-dimensional. When somebody says “a 300-dimensional embedding”, they mean a list of 300 numbers, no more mysterious than that.

Two — two vectors are equal only if every slot matches. They must have the same number of slots, and slot 1 must equal slot 1, slot 2 must equal slot 2, and so on, all the way down. So $(3, 2) \neq (2, 3)$, and $(3, 2)$ and $(3, 2, 0)$ are not equal either — they are not even the same kind of object, since one lives in $\mathbb{R}^2$ and the other in $\mathbb{R}^3$.

Three — there is a vector of all zeros. It is written $\mathbf{0}$ and it has as many slots as whatever you are working with: $(0, 0)$ in $\mathbb{R}^2$, $(0, 0, 0)$ in $\mathbb{R}^3$. Adding it to anything changes nothing, which makes it the vector world’s version of the number 0.

✓ Building a vector out of an ordinary sentence

A tea stall records Monday’s sales: *14 teas, 9 coffees, 22 samosas, 5 sandwiches*.

Step 1. Fix an order and never change it. We choose: tea, coffee, samosa, sandwich. This decision is the whole game — from now on slot 3 *means* samosas, in every vector we write for this stall.

Step 2. Write the numbers in that order. $\mathbf{m} = (14, 9, 22, 5)$.

Step 3. Name its home. Four slots, so $\mathbf{m} \in \mathbb{R}^4$. It is a four-dimensional vector, and it cannot be drawn — which does not matter in the slightest.

Step 4. Read one slot. $m_3 = 22$: twenty-two samosas. And $m_1 = 14$: fourteen teas.

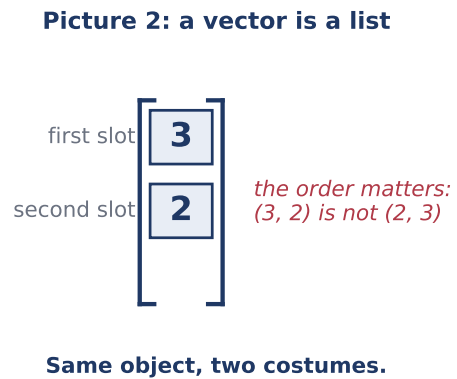
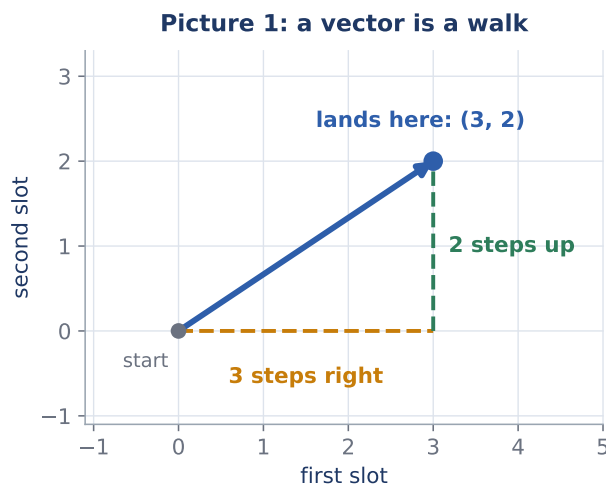
Step 5. Tuesday arrives. 11 teas, 12 coffees, 18 samosas, 7 sandwiches, so $\mathbf{t} = (11, 12, 18, 7)$. The two-day total is $\mathbf{m} + \mathbf{t} = (14 + 11, 9 + 12, 22 + 18, 5 + 7) = (25, 21, 40, 12)$.

Step 6. Six identical stalls. If a six-stall chain each sold Monday’s amounts, the chain total is $6\mathbf{m} = (84, 54, 132, 30)$.

Step 7. Sense-check. Every answer still has four slots, and slot 3 still means samosas in each one. That is the entire benefit of fixing the order in Step 1.

The picture: a vector is a walk

When a vector has only two numbers you can draw it. Read $(3, 2)$ as an instruction: *start at the corner, walk 3 steps right, then 2 steps up*. Draw an arrow from where you started to where you finished and you have drawn the vector.



How to read it. Left: the same instruction drawn as a journey. Right: the same instruction written as a list. The arrow and the list are two costumes worn by one object — use whichever helps you think.

× **Three things that trip people up**

One. A vector is not “a number with a little arrow on top”. It is several numbers kept together.

Two. A vector with 300 numbers in it cannot be drawn on paper — but it is every bit as real, and every rule in this reader still applies to it. Drawing is a teaching aid, not the definition.

Three. $(3, 2)$ and $(2, 3)$ are different vectors, exactly as house number 32 and house number 23 are different houses.

• **Remember this**

A vector is an ordered list of numbers. It can be pictured as an arrow when it has two or three components, and it exists perfectly well when it has three hundred.

2 Move one: adding two vectors

There are exactly *two* things you are allowed to do to vectors in this entire subject. Adding is the first. It is as easy as it sounds.

★ **Two people, one kitchen**

Ravi does the shopping on Monday and buys 2 kg rice, 1 kg dal and 3 litres of milk. Meera shops on Tuesday and buys 1 kg rice, 2 kg dal and no milk. How much came into the kitchen this week?

You do not need to be told the method. Rice with rice, dal with dal, milk with milk: 3 kg rice, 3 kg dal, 3 litres of milk. You matched up the slots and added within each one. That is vector addition, and you have been doing it since you were seven.

Written down, Ravi’s basket is $(2, 1, 3)$, Meera’s is $(1, 2, 0)$, and

$$\begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix} + \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} = \begin{bmatrix} 2+1 \\ 1+2 \\ 3+0 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \\ 3 \end{bmatrix}.$$

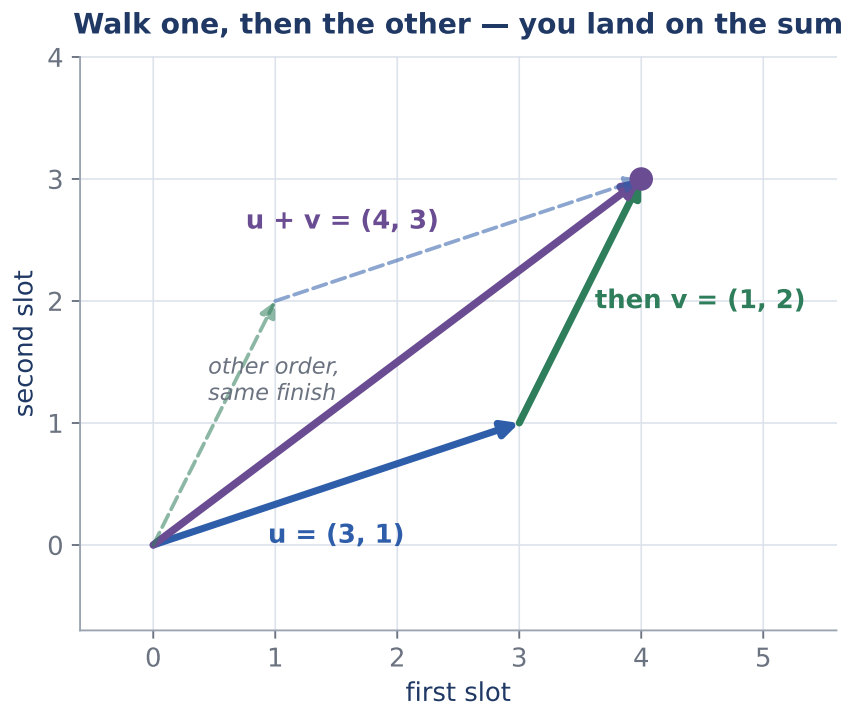
The general rule, in symbols, says the same thing:

$$\mathbf{u} + \mathbf{v} = (u_1 + v_1, u_2 + v_2, \dots, u_n + v_n).$$

Slot by slot. Rice never touches milk.

The picture: walk one journey, then the other

If adding is “do both”, the arrow picture is obvious: walk the first vector, then — from wherever you have landed — walk the second. Where you end up is the sum.



How to read it. Solid arrows: walk $\mathbf{u} = (3, 1)$ first, then $\mathbf{v} = (1, 2)$ from there. The purple arrow straight from start to finish is $\mathbf{u} + \mathbf{v} = (4, 3)$. The faint dashed arrows do it in the opposite order and land in the same place — which is why $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$.

✓ Adding, step by step

Add $\mathbf{u} = (5, -2, 4)$ and $\mathbf{v} = (-1, 7, 0)$.

Step 1. Slot one. $5 + (-1) = 4$.

Step 2. Slot two. $-2 + 7 = 5$.

Step 3. Slot three. $4 + 0 = 4$.

Step 4. Collect. $\mathbf{u} + \mathbf{v} = (4, 5, 4)$.

Step 5. Sense-check. The answer still has three slots, exactly like the two vectors we started with. Adding never changes the length of the list.

✗ Lengths must match

You cannot add $(1, 2, 3)$ to $(4, 5)$. There is no third number in the second list for the 3 to pair up with, and inventing a zero would be a decision nobody authorised. In this subject, “the shapes do not match” is the single most common error message you will ever see from a computer, and this is where it starts.

3 Move two: scaling a vector

The second and last permitted move is **scaling**: multiplying a whole vector by one ordinary number.

★ Doubling a recipe

A recipe for four people needs 2 cups of rice, 1 cup of dal and 3 spoons of ghee — the vector $(2, 1, 3)$. Eight people are coming. You double everything: $(4, 2, 6)$. Cooking for two instead? Halve everything: $(1, 0.5, 1.5)$.

You never doubled just the rice. The single number multiplies every slot. That is scaling.

In symbols, for any plain number λ (the Greek letter *lambda*, used throughout this subject for exactly this job):

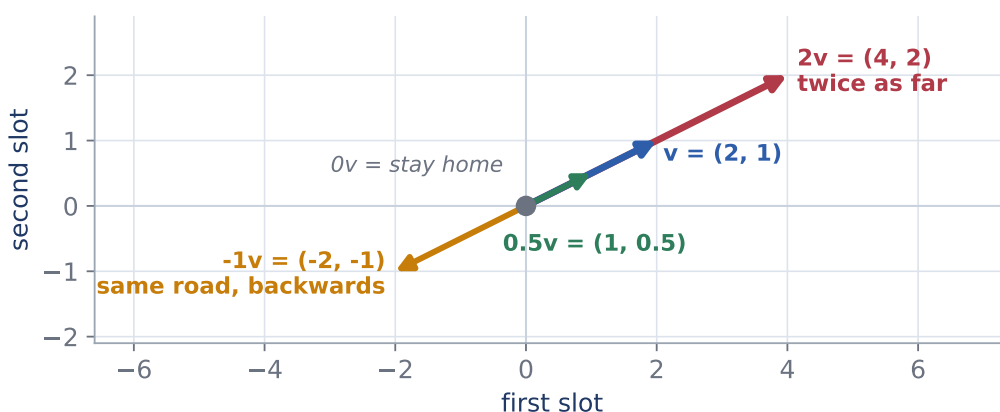
$$\lambda \mathbf{v} = (\lambda v_1, \lambda v_2, \dots, \lambda v_n).$$

That plain number has a name: a **scalar**. It is called a scalar because scaling is what it does — it stretches or shrinks the vector without changing what the slots mean.

Four cases are worth naming out loud:

- $\lambda > 1$ — same direction, longer. $2\mathbf{v}$ goes twice as far.
- $0 < \lambda < 1$ — same direction, shorter. $0.5\mathbf{v}$ stops halfway.
- $\lambda < 0$ — *backwards* along the same road. $-1\mathbf{v}$ is the return journey.
- $\lambda = 0$ — you do not move at all. You get the **zero vector** $(0, 0, \dots, 0)$, which is the “stay at home” vector.

Scaling stretches or flips — the direction stays on one line



How to read it. Every scaled copy of $\mathbf{v}$ lies on one straight line through the origin. Scaling changes how far you travel and possibly whether you go forwards or backwards, but never the road you travel on.

✓ Scaling, step by step

Compute $-3\mathbf{v}$ where $\mathbf{v} = (2, -1, 4)$.

Step 1. $-3 \times 2 = -6$.

Step 2. $-3 \times (-1) = 3$.

Step 3. $-3 \times 4 = -12$.

Step 4. $-3\mathbf{v} = (-6, 3, -12)$.

Step 5. Sense-check. Every entry is three times as big as before, and every sign has flipped. Three times as far, opposite direction. That is what -3 should do.

4 Putting the two moves together

Scaling and adding are the only two moves, so anything you can build with them is worth a name. Scale some vectors, then add the results, and what you get is called a **linear combination** of them:

$$\lambda_1 \mathbf{v}_1 + \lambda_2 \mathbf{v}_2 + \cdots + \lambda_k \mathbf{v}_k.$$

★ A recipe made out of recipes

You have a standard *dal* portion and a standard *rice* portion, each written as a vector of ingredients. A thali for the family is “2 dal portions plus 3 rice portions” — scale each one, add them up, and read off the total ingredients. Every mixture you can make from those two portions is a linear combination of them.

That phrase — *linear combination* — will appear on nearly every page from here on. Whenever you see it, translate it in your head as: *scale some things, then add them up*.

Closure: the two moves never take you outside

Here is a quiet but important fact. Add two lists of three numbers and you get a list of three numbers. Scale a list of three numbers and you get a list of three numbers. You cannot escape. Whatever you do with adding and scaling, the answer is still the same kind of object you started with.

That property has a name: **closure**. A set of objects that is closed under adding and scaling is called a **vector space**.

× Closure can fail, and here is a set where it does

Consider the set of all vectors in $\mathbb{R}^2$ whose *first* number is exactly 1: things like $(1, 5)$ and $(1, -2)$. Add two of them: $(1, 5) + (1, -2) = (2, 3)$. The first number is now 2, so the answer has fallen out of the set. That set is *not* closed, so it is not a vector space. Closure is a real condition, not a formality.

Vectors do not have to look like lists

The lecture makes a point that sounds abstract but is worth pinning down. *Anything* you can add and scale sensibly counts as a vector. Take polynomials:

$$(2 + x) + (1 + 3x) = 3 + 4x.$$

Now write each polynomial as the list of its coefficients — the constant first, the x part second:

$$(2, 1) + (1, 3) = (3, 4).$$

Identical arithmetic. The polynomial was a vector wearing a different costume all along. The same is true of images, sound clips and many other things. In this course we stay with plain lists in $\mathbb{R}^n$, but it is useful to know the idea is bigger than the notation.

• Remember this

Two moves, and only two: add slot-by-slot, and scale everything by one number. Doing them together is called taking a linear combination. Closure means those moves never throw you out of the set you started in — and a set with that guarantee is called a vector space.

5 From vectors to matrices: stack the lists

So far, one flat has been one vector. But you never look at one flat. You look at a whole list of them, and you write them one under another — which is to say, you make a table.

★ A class register

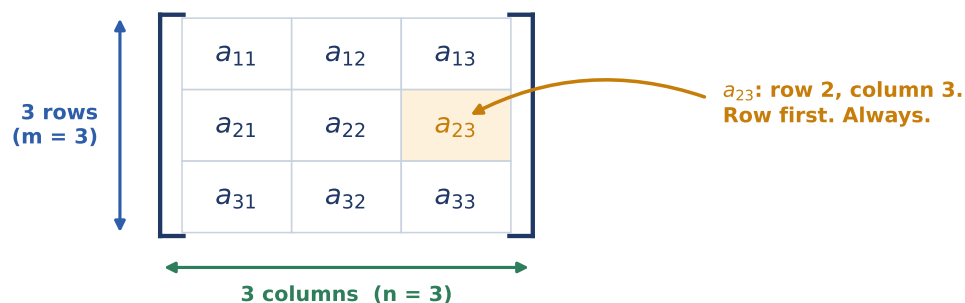
One student's marks in three subjects is a vector: $(72, 65, 88)$. Now write the whole class down, one student per line, subjects always in the same order across the page. You have just built a table: 40 rows for 40 students, 3 columns for 3 subjects.

Every school in the country keeps this table. Mathematics calls it a **matrix**.

A **matrix** is a rectangular grid of numbers: m rows tall and n columns wide, written $A \in \mathbb{R}^{m \times n}$. Say the size out loud as “ m by n ”, and always say the rows first. A grid with 40 students and 3 subjects is a 40×3 matrix.

To pick out a single number you give its row and then its column: a_{ij} is the entry in row i , column j .

How to find any entry in a grid



House number style: the floor, then the flat on that floor.

How to read it. a_{23} means row 2, column 3 — never the other way round. The habit is the same one you already use for an address: the floor first, then the flat on that floor.

✓ Reading a real table as a matrix

Three shops, two products, units sold last week:

	Notebooks	Pens
Shop 1	40	120
Shop 2	25	90
Shop 3	60	30

Step 1. Strip the words. Keep the numbers in place: $A = \begin{bmatrix} 40 & 120 \\ 25 & 90 \\ 60 & 30 \end{bmatrix}$.

Step 2. State the shape. Three rows, two columns, so $A \in \mathbb{R}^{3 \times 2}$. A “3 by 2” matrix.

Step 3. Find an entry. a_{32} means row 3, column 2 — Shop 3, pens — which is 30.

Step 4. Read a whole row. Row 2 is (25, 90): everything Shop 2 sold. One row is one shop’s vector.

Step 5. Read a whole column. Column 1 is (40, 25, 60): notebook sales across all three shops. One column is one product’s vector.

Vectors are just very thin matrices

Two special shapes have their own names, and they connect the last few sections to this one:

- A **row vector** is a $1 \times n$ matrix — one row, several columns.
- A **column vector** is an $m \times 1$ matrix — several rows, one column.

So a vector is not a new species. It is a matrix that happens to be one cell wide (or one cell tall). Everything true of matrices is therefore automatically true of vectors, which saves us proving anything twice.

Two honest ways to read the same grid

This point will pay for itself several times over. A matrix can be read as:

- a **stack of rows** — each row is one record, one shop, one student, one clue; or
- a **row of columns** — each column is one quantity measured across every record.

Neither reading is more correct. They are two ways of holding the same object, and later (Part IV) we will solve the same problem twice, once with each reading, and get the same answer with quite different-feeling arithmetic.

× **Row first, column second — always**

a_{23} and a_{32} are different numbers sitting in different places. This never varies, in any book, in any software library. Fix it now and it will never bother you again.

• **Remember this**

A matrix is a grid: m rows, n columns, entry a_{ij} at row i and column j . Stack vectors and you get a matrix; slice a matrix and you get vectors. They are the same family.

6 A gallery of matrices you already know

Before going further it is worth seeing how ordinary matrices are. In every example below, nobody invented the grid — it was already the natural way to write the information down.

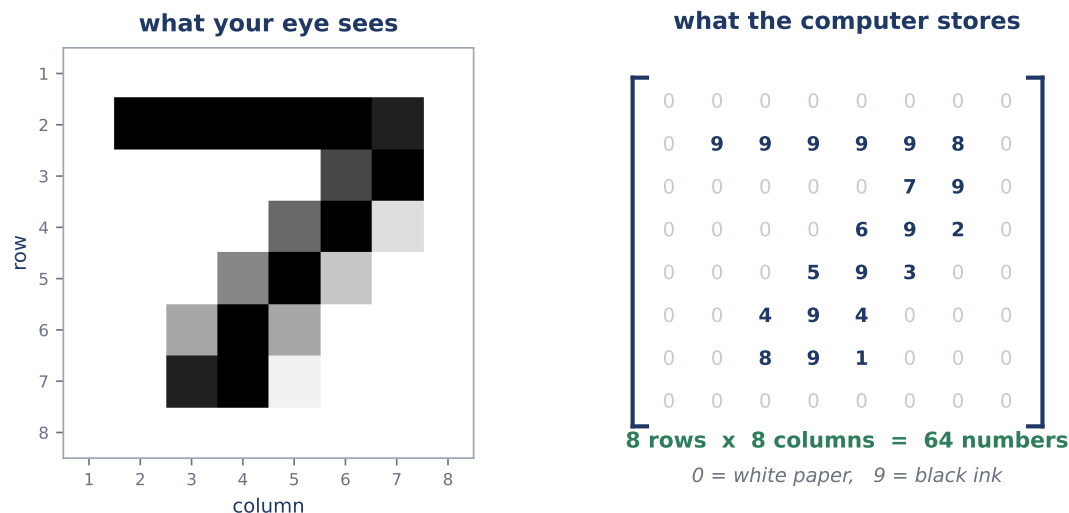
One: any table with rows and columns

The class register from the last section. The sales figures for four shops across three products. A cricket scorecard with batsmen down the side and runs, balls, fours and sixes across the top.

Anything you would put in a spreadsheet is a matrix the moment you delete the headings.

Two: a photograph

This one surprises people. A black-and-white photograph is a grid of tiny squares, and each square holds one number saying how dark it is. That grid is a matrix, exactly as defined.



How to read it. The picture on the left and the grid on the right are the same object. Every square of the picture is one entry of the matrix. Here 0 means white and 9 means black; a real image usually runs 0 to 255.

A colour photograph is three such matrices stacked — one for red, one for green, one for blue. A photo from your phone at 4000×3000 pixels is therefore three matrices of twelve million numbers each. When you hear that image processing is “just linear algebra”, this is what is meant, and it is meant literally.

Three: a table of distances

Road distances between four cities, in kilometres:

	Delhi	Jaipur	Agra	Lucknow
Delhi	0	280	230	550
Jaipur	280	0	240	800
Agra	230	240	0	330
Lucknow	550	800	330	0

Two things stand out, and both have names. The diagonal is all zeros, because every city is zero kilometres from itself. And the grid is a mirror image of itself across that diagonal — Delhi to Jaipur is the same 280 km as Jaipur to Delhi. A matrix with that mirror property is called **symmetric**, and it satisfies $a_{ij} = a_{ji}$ for every i and j .

Four: a single vector

A shopping list $(2, 1, 3)$ written as a column is a 3×1 matrix. Written across the page it is a 1×3 matrix. As we said: vectors are simply very thin matrices, so everything in this gallery applies to them too.

The handful of shapes worth naming

You will meet these words constantly, and each one is just a description of what the grid looks like:

Name	What it means	A small example
rectangular	rows and columns differ in number	$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ (2×3)
square	the same number of rows as columns	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ (2×2)
zero	every entry is 0	$\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$
diagonal	only the top-left-to-bottom-right line is non-zero	$\begin{bmatrix} 5 & 0 \\ 0 & 2 \end{bmatrix}$
identity I	diagonal, with 1s on the diagonal	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$
symmetric	a mirror image of itself across the diagonal	$\begin{bmatrix} 1 & 7 \\ 7 & 3 \end{bmatrix}$

Only square matrices can be diagonal, identity or symmetric — the definitions do not even make sense otherwise. And two of these will matter a great deal later: the identity turns out to be the “do nothing” matrix (Part III), and only square matrices can ever have an inverse.

× A matrix is not automatically square

Beginners often picture matrices as neat squares, because textbook examples usually are. Real ones almost never are. A dataset of 50,000 customers with 12 measurements each is a $50,000 \times 12$ matrix — very tall and very thin. That is the normal case.

• Remember this

Spreadsheets, photographs, distance tables and shopping lists are all matrices already. The words square, diagonal, identity and symmetric are just descriptions of what the grid looks like — nothing more.

7 What people actually mean by “data”

The word *data* gets used loosely. In this subject it has a precise and very simple meaning, and it is worth nailing down because every textbook, library and colleague will use half a dozen different words for the same three things.

★ Four second-hand scooters

You are trying to work out what makes a used scooter expensive. For each scooter you write down its age, the kilometres it has run, and how many owners it has had — and, because they have already been sold, you also know the price each one fetched.

That is a dataset. Four scooters, three measurements each, plus the answer you care about.

	age	km run	owners	price
Scooter A	2	12	1	48
Scooter B	5	30	2	30
Scooter C	1	6	1	55
Scooter D	7	44	3	22

$m = 4$ rows (how many examples you have) $n = 3$ feature columns (how many things you measured)

How to read it. One table, three parts, and a pile of synonyms for each part. The names differ by textbook and by industry; the objects never do.

Here is the same information as a straight translation table, because the synonyms are the part that genuinely confuses beginners:

The object	What it is	Other names you will hear
one row	everything you measured about one thing	record, example, observation, sample, instance, data point
one column	one measurement, taken across every thing	feature, attribute, variable, predictor, input, covariate
the target	the answer you want to predict	label, output, response, ground truth, dependent variable, y
m	how many rows you have	sample size, number of examples
n	how many feature columns you have	number of features, dimensionality

Stack the feature columns and you have the **data matrix** X , which is $m \times n$. Stack the targets and you have the **target vector** y , which has m entries. That is the entire vocabulary.

✓ Turning a paragraph of words into X and y

“Three second-hand scooters sold last month. The first was 2 years old, had run 12 thousand km, and went for Rs. 48,000. The second was 5 years old with 30 thousand km and went for Rs. 30,000. The third was 1 year old with 6 thousand km and fetched Rs. 55,000.”

Step 1. Decide what is a feature and what is the target. Age and kilometres are things you can measure before selling — features. Price is what you want to predict — the target.

Step 2. Fix a column order. Age first, kilometres second. Now stick to it.

Step 3. Write one row per scooter, in that order.

$$X = \begin{bmatrix} 2 & 12 \\ 5 & 30 \\ 1 & 6 \end{bmatrix}, \quad y = \begin{bmatrix} 48 \\ 30 \\ 55 \end{bmatrix} \quad (\text{prices in thousands of rupees}).$$

Step 4. State the shape. X is 3×2 : three examples, two features. So $m = 3$ and $n = 2$. And

$\mathbf{y}$ has 3 entries — always one per row of X .

Step 5. Read a row. Row 2 is $(5, 30)$: the second scooter, five years old, thirty thousand kilometres. Its target is $y_2 = 30$.

Step 6. Read a column. Column 1 is $(2, 5, 1)$: the ages of all three scooters. This is the vector you would use to answer “how old are these scooters on average?”

Step 7. Sense-check. Rows and $\mathbf{y}$ have to line up. If X has 3 rows and $\mathbf{y}$ has 4 entries, something has gone wrong in the bookkeeping — one scooter has lost its price.

What if a column is not a number?

Real tables are full of words: a city name, a yes/no, a date, a size of “small / medium / large”. Matrices hold numbers only, so these have to be converted first. The standard moves are simple:

- **Yes/no becomes 1/0.** “Has insurance: yes” becomes a column holding 1; “no” becomes 0. One column, two possible values.
- **A category becomes one column per value.** A city column holding Delhi, Mumbai or Pune becomes *three* columns — “is Delhi”, “is Mumbai”, “is Pune” — and each row has exactly one 1 among them. This is called **one-hot encoding**, because in every row exactly one of the new columns is hot.
- **A date becomes a count.** “Bought on 14 March” becomes “days since the start of the year”, or “months old” — some number you can do arithmetic with.
- **An ordered scale becomes ordered numbers.** Small / medium / large can legitimately become 1 / 2 / 3, because those really are in order and the gaps are meaningful.

✓ Turning a table of words into a numeric X

	City	Insured	Age
Scooter A	Delhi	yes	2
Scooter B	Pune	no	5
Scooter C	Mumbai	yes	1
Scooter D	Delhi	no	7

Step 1. Age is already a number. Leave it alone: $(2, 5, 1, 7)$.

Step 2. Insured is yes/no. One column, 1 for yes and 0 for no: $(1, 0, 1, 0)$.

Step 3. City has three possible values, so it becomes three columns — is-Delhi, is-Mumbai, is-Pune — with a single 1 in each row: Scooter A is $(1, 0, 0)$, B is $(0, 0, 1)$, C is $(0, 1, 0)$, D is $(1, 0, 0)$.

Step 4. Assemble, column order fixed as (is-Delhi, is-Mumbai, is-Pune, insured, age).

$$X = \begin{bmatrix} 1 & 0 & 0 & 1 & 2 \\ 0 & 0 & 1 & 0 & 5 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 7 \end{bmatrix}$$

Step 5. Note what happened to the shape. Three written columns became five numeric ones, so X is 4×5 . The number of *rows* never changed — there are still four scooters. Encoding widens a table; it never lengthens it.

× **Never number your categories 1, 2, 3**

It is tempting to write Delhi = 1, Mumbai = 2, Pune = 3 and keep one column. Do not. Arithmetic on that column would claim that Pune is three times Delhi, and that Mumbai sits exactly halfway between them. Neither statement means anything. Cities have no order and no spacing, so they need one column each. Sizes small/medium/large *do* have an order, so there 1/2/3 is fine — the test is always whether the ordering and the gaps are real.

• **Remember this**

A dataset is a matrix X of features (one row per thing, one column per measurement) and a vector y of the answers. Row, record, example and observation all mean the same thing; so do column, feature and variable. Words in a table must be turned into numbers before any of this works.

8 Why machine learning is built out of exactly these two things

Everything above may still feel like tidy bookkeeping. This section is where it turns into the reason the subject exists.

Your data is already a matrix

Any dataset you have ever opened — a spreadsheet, a CSV file, a database export — has rows and columns. One row per thing you measured. One column per property you measured about it. That is a matrix, and nobody had to convert anything.

the spreadsheet you were given

size in hundreds of sq ft, age in years

	size	rooms	age		
				the matrix X 4 rows x 3 columns	the answers y monthly rent
Flat A	12	3	8	same thing →	42 28 55 20
Flat B	9	2	15		
Flat C	15	3	2		
Flat D	7	1	20		

one row = one flat = one vector (9, 2, 15)

one column = one feature across all flats (12, 9, 15, 7)

How to read it. The spreadsheet on the left and the matrix in the middle are the same object with the borders removed. One row (amber) is one flat — a vector. One column (green) is one feature measured across every flat — also a vector. The purple column on the right holds the thing you are trying to predict.

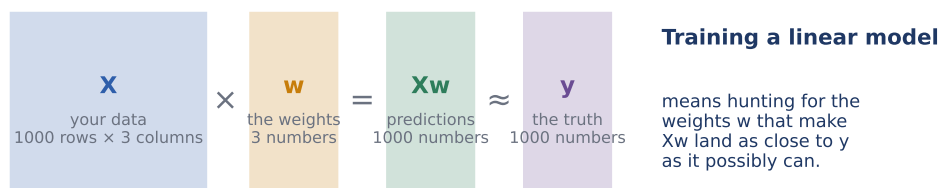
The standard names, which you will meet in every textbook and every library:

- X is the **data matrix** or *feature matrix*: m rows (one per example) and n columns (one per feature). Here X is 4×3 .
- y is the **target vector**: the correct answer for each row. Here it has 4 entries, one rent per flat.
- w is the **weight vector**: how much each feature matters. It has n entries — one weight per column of X — and it is the thing you do *not* know.

A prediction is one multiplication

Suppose you decide that rent depends on size and bedrooms like this: multiply the size by some number w_1 , multiply the bedrooms by some number w_2 , add the two results. For one flat that is a handful of multiplications and one addition.

Now do it for all four flats at once, and then for all four million. Written out longhand it is unbearable. Written as a matrix touching a vector it is three characters: $X\mathbf{w}$.



X and $\mathbf{y}$ are handed to you. $\mathbf{w}$ is the unknown — exactly like the kit counts in the workshop.

How to read it. X and $\mathbf{y}$ come from the world and are fixed. $\mathbf{w}$ is the unknown. $X\mathbf{w}$ is what your current guess at $\mathbf{w}$ predicts. Training means moving $\mathbf{w}$ until $X\mathbf{w}$ sits as close to $\mathbf{y}$ as it can.

ML The one sentence this whole session is building towards

Almost every model you will meet — linear regression, logistic regression, a neural network layer, the attention step inside a language model — computes its answer by multiplying a matrix by a vector and then doing something small afterwards. Learn to read $X\mathbf{w}$ fluently and you can read the first line of nearly any model.

Three reasons this framing wins

One: it is compact. $X\mathbf{w}$ says the same thing as a page of loops, and you can hold it in your head.

Two: it is fast. Graphics cards are physically built to multiply matrices. The same arithmetic written as a loop over rows can be hundreds of times slower — which is a large part of why modern machine learning became possible at all.

Three: it turns learning into solving. Once the model is $X\mathbf{w} \approx \mathbf{y}$, the question “what should the model learn?” has become “which $\mathbf{w}$ satisfies these equations?” — and that is a question with a real, reliable method behind it. The rest of this reader is that method.

And almost anything can be a vector

The reason this covers so much ground is that data of every kind gets turned into lists of numbers before a model ever sees it:

- A photograph is a long list of pixel brightnesses. A small 28×28 grey image is a vector with 784 entries.
- A word is turned into a list of a few hundred numbers, called an *embedding*, chosen so that words used in similar ways get similar lists.
- A customer is a list: how many orders, days since last visit, average basket size.
- A sound clip is a list of air-pressure samples, tens of thousands of them per second.

Different worlds, one shape. And because they all arrive as vectors, one toolbox serves all of them.

• Remember this

Rows of your dataset are vectors; the dataset itself is a matrix X ; the thing the model must learn is a vector $\mathbf{w}$; and a prediction for every row at once is the single expression $X\mathbf{w}$. That is why this subject comes first in a machine learning degree.

Part I on one page

Here is everything Part I established, in the order you met it. If any line below feels unfamiliar, that is the section worth rereading before going on.

• The six things to carry into Part II

- A **vector** is an ordered list of numbers. Its *dimension* is simply how many numbers are in the list.
- There are exactly two things you may do to vectors: **add** them slot by slot, and **scale** them by one plain number. Doing both at once is a **linear combination**.
- Those two moves never take you outside the set you started in — that is **closure**, and a set with that guarantee is called a **vector space**.
- A **matrix** is a grid, m rows by n columns, with a_{ij} meaning row i , column j . Spreadsheets, photographs and distance tables are already matrices, and a vector is just a matrix one cell wide.
- Your **data** is a matrix X — one row per example, one column per feature — together with a target vector $\mathbf{y}$. Anything recorded in words has to be turned into numbers first.
- A model's guesses for every row at once are $X\mathbf{w}$, where $\mathbf{w}$ holds the weights the model has to learn. Finding $\mathbf{w}$ is what the rest of this reader is about.

Part II · Clues about hidden numbers

What a linear equation really is, and the three fates that can befall a set of them

9 Start here: a canteen bill

Forget matrices for a moment. Here is a puzzle you can solve without any new mathematics at all.

★ Two bills, two unknown prices

The office canteen has stopped printing prices on the menu. You want to know what a tea costs and what a samosa costs.

Yesterday Anjali bought **2 teas and 3 samosas** and paid **Rs. 65**.

Today Vikram bought **1 tea and 4 samosas** and paid **Rs. 70**.

Neither bill tells you a price on its own. Together, they do.

Give the unknown prices names. Let t be the price of one tea and s the price of one samosa. Each bill becomes a sentence in symbols:

$$2t + 3s = 65 \quad (\text{Anjali's bill}) \quad (1)$$

$$t + 4s = 70 \quad (\text{Vikram's bill}) \quad (2)$$

Two sentences, two unknowns. Each one is a **linear equation**. The pair of them is a **system of linear equations**. Solving the system means finding numbers for t and s that make *both* sentences true at the same time — not one, both.

✓ Cracking the canteen, two different ways

Method A — substitute.

Step 1. Rearrange (2) to get t on its own: $t = 70 - 4s$.

Step 2. Put that into (1) wherever t appears: $2(70 - 4s) + 3s = 65$.

Step 3. Open the bracket: $140 - 8s + 3s = 65$, so $140 - 5s = 65$.

Step 4. Tidy: $-5s = -75$, therefore $s = 15$.

Step 5. Back into Step 1: $t = 70 - 4(15) = 70 - 60 = 10$.

Method B — make one unknown cancel. (Remember this one. It grows up into the main method of this whole session.)

Step 1. Double every part of (2): $2t + 8s = 140$. This is still a true statement about the same prices — doubling both sides of a true equation keeps it true.

Step 2. Subtract (1) from it. The $2t$ terms cancel exactly: $(2t + 8s) - (2t + 3s) = 140 - 65$, which leaves $5s = 75$.

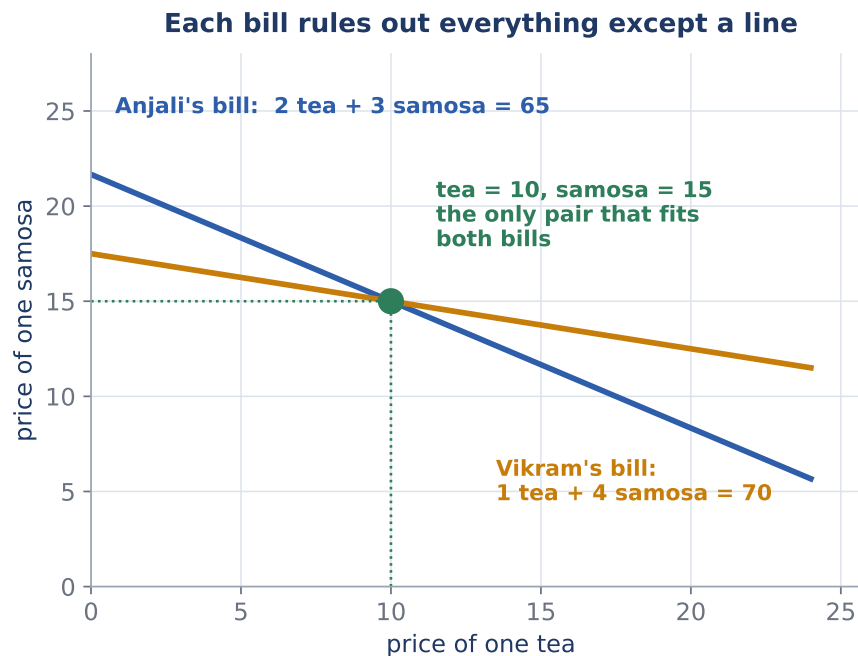
Step 3. So $s = 15$, and (2) gives $t = 70 - 60 = 10$.

Check, always. Tea 10, samosa 15. Anjali: $2(10) + 3(15) = 20 + 45 = 65$. ✓ Vikram: $1(10) + 4(15) = 10 + 60 = 70$. ✓ Both bills agree, so the prices are right.

Take a moment with Method B, because it is the seed of everything ahead. We did two things to the equations: we *multiplied one by a number*, and we *subtracted one from another*. Neither move changed which prices are correct — it only made the arithmetic easier. That idea, applied over and over in a fixed order, is the whole of Gaussian elimination in Part IV.

The picture

Anjali's bill by itself does not pin down the prices — lots of price pairs give a total of 65. But not that many. If you draw every pair (t, s) that satisfies her bill, you get a straight line. Do the same for Vikram's bill and you get a different straight line. The one pair that satisfies both is where the lines cross.



How to read it. Each bill rules out the entire plane except one line. Two lines, one crossing point, one answer: tea 10, samosa 15.

Remember this

A linear equation is a *clue* about numbers you cannot see. A system is several clues that must all be true together. Solving means finding the numbers that satisfy every clue at once — and you should always check your answer against every original clue.

10 What “linear” actually means

Look again at $2t + 3s = 65$. Notice what is happening to the unknowns, and — more importantly — what is *not*.

Each unknown is multiplied by a number you already know (2 and 3), and the results are added. Nothing else touches t or s . That is the whole rule.

► The rule in one line

Known number times unknown, added up. An unknown may be multiplied by a number you already know, and the pieces may be added together. Anything else is not allowed.

So these are linear equations:

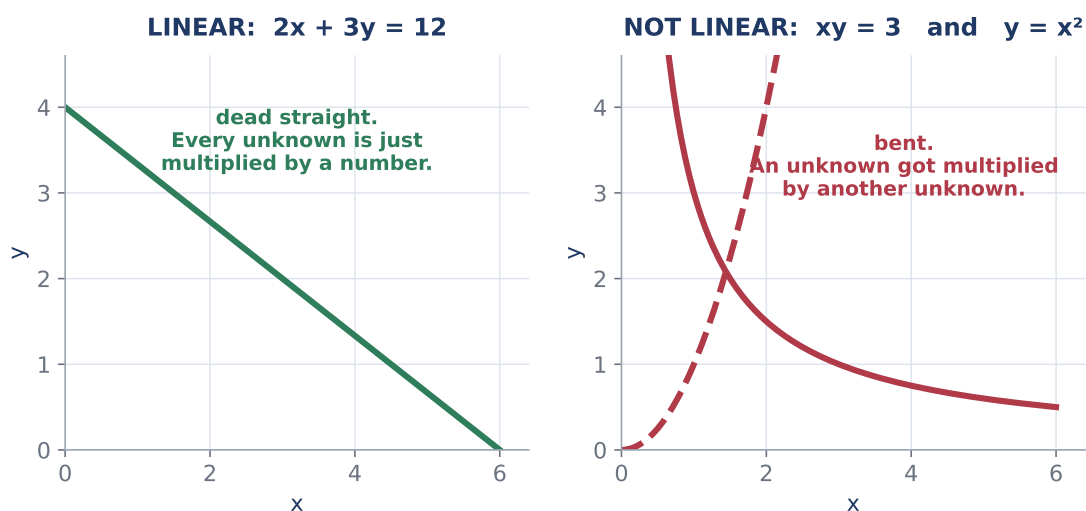
$$3x_1 - 7x_2 = 4, \quad x_1 + x_2 + x_3 = 0, \quad 0.5a - 2b + 9c - d = 12.$$

And these are not, each for its own reason:

Not allowed	Why	Read it as
x^2	unknown times itself	$x \cdot x$ — two unknowns multiplied
$x_1 x_2$	unknown times another unknown	neither factor is known
$1/x$	unknown in the denominator	dividing <i>by</i> something unknown
$\sqrt{x}$	unknown under a root	not a plain multiple of x
$\sin x$	unknown inside a function	bends the relationship
2^x	unknown as an exponent	grows explosively, not steadily

× “Linear” bans much more than squares

Most people remember that x^2 is not allowed and stop there. But $x_1 x_2$ is just as forbidden, and it is far easier to miss because neither symbol is squared. The test is not “is anything squared?” — it is **“is every unknown multiplied only by a number I already know?”** If two unknowns are multiplied together, or an unknown is divided by, or an unknown sits inside a function, the equation is not linear.



How to read it. Linear equations draw perfectly straight lines — which is where the name comes from. The moment an unknown multiplies another unknown, or itself, the picture bends.

Why we care so much about this restriction

Straight things are solvable. For linear systems there is a procedure that always works: it finds the answer if one exists, and proves none exists if it does not. It never gets stuck and never needs a lucky guess. Once equations start bending, no such universal procedure exists, and mathematics gets much harder very fast.

So linear algebra is not a simplified version of the real thing. It is the part of the subject where complete, reliable answers are available — which is exactly why so much of machine learning is built to stay inside it, or to approximate hard problems with linear ones.

× Linear does not mean small

An equation with 500 unknowns is still linear if every unknown is only multiplied by a known number. “Linear” describes the *shape* of the relationship, not the size of the problem.

11 A bigger example: the workshop

The canteen had two unknowns. Here is the lecture's own example, which has three — and a few more clues.

★ A workshop that wants nothing left over

A small furniture workshop makes flat-pack kits of three kinds: **chair kits**, **table kits** and **cabinet kits**.

Making any kit consumes four things: some *wood*, some *labour hours*, some *machine time*, and some *shipping effort*. Different kits consume different amounts — a table kit uses more wood than a chair kit, a chair kit takes more labour, and so on.

This week the workshop has a fixed stock of each of those four things. The owner wants to finish the week having used up **every last unit** of all four — nothing left sitting in the store room, and nothing running out early.

The question is: how many kits of each type should they build?

Give the three unknowns names, exactly as we named t and s for the canteen:

x_1 = number of chair kits, x_2 = number of table kits, x_3 = number of cabinet kits.

Here is what each kit consumes, and how much of each resource is in stock:

Resource	Chair kit	Table kit	Cabinet kit	In stock
wood	1	2	1	9
labour hours	2	1	1	8
machine time	1	1	2	7
shipping	3	1	2	11

Now read the first row out loud, slowly, in ordinary words:

Each chair kit eats 1 unit of wood. Each table kit eats 2. Each cabinet kit eats 1. Add up all the wood eaten, and it has to come to exactly 9 — the amount in stock.

Turn that sentence into symbols and you get $x_1 + 2x_2 + x_3 = 9$. One resource, one clue. Do the same for the other three rows:

$$\begin{array}{llll}
 x_1 + 2x_2 + x_3 = 9 & \text{(wood)} & (3) \\
 2x_1 + x_2 + x_3 = 8 & \text{(labour)} & (4) \\
 x_1 + x_2 + 2x_3 = 7 & \text{(machine)} & (5) \\
 3x_1 + x_2 + 2x_3 = 11 & \text{(shipping)} & (6)
 \end{array}$$

Four clues, three unknowns. Notice that the table you were handed *is* the system: each row of the table became a row of equations, and the “in stock” column became the numbers on the right. Nothing was invented.

The lecture tells us the answer is $x_1 = 2$, $x_2 = 3$, $x_3 = 1$. Never take that on trust — check it.

✓ Does the plan (2 chairs, 3 tables, 1 cabinet) really clear the store room?

Put $x_1 = 2$, $x_2 = 3$, $x_3 = 1$ into all four clues.

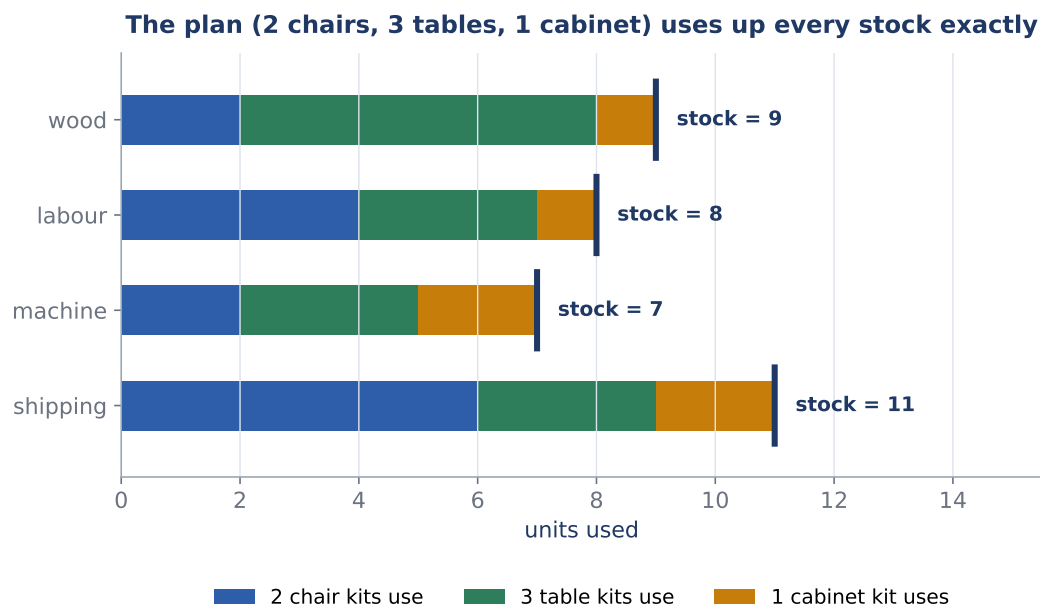
Step 1. Wood. $1(2) + 2(3) + 1(1) = 2 + 6 + 1 = 9$. Stock is 9. ✓

Step 2. Labour. $2(2) + 1(3) + 1(1) = 4 + 3 + 1 = 8$. Stock is 8. ✓

Step 3. Machine. $1(2) + 1(3) + 2(1) = 2 + 3 + 2 = 7$. Stock is 7. ✓

Step 4. Shipping. $3(2) + 1(3) + 2(1) = 6 + 3 + 2 = 11$. Stock is 11. ✓

Step 5. Verdict. All four land exactly on their stock levels at the same time, so $(2, 3, 1)$ solves the system. Build 2 chair kits, 3 table kits and 1 cabinet kit, and the store room ends the week empty.



How to read it. Each bar shows what the plan consumes of one resource, split by which kit consumed it. The navy tick is the stock level. Every bar stops precisely on its tick — which is the picture of “all four clues satisfied at once”.

12 The same idea, but now it is machine learning

The workshop example is not a detour. Swap two words and it becomes the central problem of supervised learning. Let us do the swap slowly, because this is the connection the whole course rests on.

	In the workshop	In machine learning
The unknowns	how many kits of each type to build	how much each feature matters — the <i>weights</i>
One clue	one resource: what it costs per kit, and how much is in stock	one record from your data: its feature values, and its known answer
Solving the system	find a production plan that fits every resource	find weights that reproduce every recorded answer

Here is the same idea with actual numbers.

★ **Learning what drives rent**

You believe monthly rent depends on two things: the size of the flat and the number of bedrooms. You do not know how much each one contributes — that is precisely what you want the data to tell you.

Write w_1 for the rupees-per-unit-of-size and w_2 for the rupees-per-bedroom. These are your unknowns, exactly like the kit counts.

You know the rent of two flats already. Each known flat is one clue.

✓ **Two flats, two unknowns, one model**

Sizes are in hundreds of square feet and rents in thousands of rupees, to keep the numbers small.

	Size	Bedrooms	Rent (known)
Flat A	8	2	28
Flat B	10	4	38

Step 1. Turn each flat into a clue. Flat A says: size 8 times its weight, plus 2 bedrooms times their weight, must come to the rent of 28.

$$8w_1 + 2w_2 = 28, \quad 10w_1 + 4w_2 = 38.$$

Compare this with the wood row of the workshop. It is the same sentence with different nouns.

Step 2. Simplify the first clue. Divide it through by 2: $4w_1 + w_2 = 14$, so $w_2 = 14 - 4w_1$.

Step 3. Substitute into the second clue. $10w_1 + 4(14 - 4w_1) = 38$, that is $10w_1 + 56 - 16w_1 = 38$.

Step 4. Tidy. $-6w_1 = -18$, so $w_1 = 3$.

Step 5. Back-substitute. $w_2 = 14 - 4(3) = 2$.

Step 6. Check both clues. Flat A: $8(3) + 2(2) = 24 + 4 = 28$. ✓ Flat B: $10(3) + 4(2) = 30 + 8 = 38$. ✓

Step 7. Read the answer in words. The learned model is

$$\text{rent} = 3 \times \text{size} + 2 \times \text{bedrooms}.$$

Every extra hundred square feet is worth Rs. 3,000 a month; every extra bedroom is worth Rs. 2,000.

Step 8. Use it. A new flat of 900 sq ft with 3 bedrooms: $3(9) + 2(3) = 27 + 6 = 33$, so about Rs. 33,000 a month. You have just made a prediction from a model you fitted by solving a system of linear equations.

That is a complete, honest machine learning workflow in eight steps. Real problems differ in size, not in kind — and one thing more, which matters and which we will meet head-on later:

ML The one thing real data does differently

Real datasets do not give you two clues for two unknowns. They give you *thousands* of clues for a handful of unknowns. With 1000 flats and 2 weights, there is almost never a (w_1, w_2) that satisfies all 1000 clues exactly — one flat has a great view, another has a bad landlord, and no straight-line rule can capture that.

So the system has **no exact solution**. That sounds like failure. It is not: it is where *linear regression* begins. Instead of demanding an exact fit, we look for the weights that come closest overall. Section 28 at the end of this reader comes back to it, once we have the tools.

13 How many answers can a system have?

Here is a fact that surprises most people the first time: a system of linear equations has either **no answer**, **exactly one answer**, or **infinitely many answers**. There is no fourth option. A linear system can never have exactly two answers, or exactly seventeen.

Why “exactly two” is impossible

Suppose two different answers both work. Then their halfway point works too — and so does every point on the straight line joining them. So the moment you have two, you have endless. Here is that argument made concrete.

✓ **Two answers force a third, and a fourth, and so on**

Take the single clue $x_1 + x_2 = 10$. Two obvious answers: $(4, 6)$ and $(8, 2)$.

Step 1. Average them. Halfway between is $(\frac{4+8}{2}, \frac{6+2}{2}) = (6, 4)$.

Step 2. Test it. $6 + 4 = 10$. ✓ A third answer, for free.

Step 3. Try any other blend. Take one quarter of the first and three quarters of the second: $(0.25 \times 4 + 0.75 \times 8, 0.25 \times 6 + 0.75 \times 2) = (7, 3)$. Test: $7 + 3 = 10$. ✓

Step 4. See why it always works. Both answers push the left-hand side to exactly 10. Any blend of them pushes it to a blend of 10 and 10 — which is 10. The clue cannot tell the blends apart.

Step 5. Conclude. There is no way to stop at two. Two answers means an entire line of answers.

The three cases, in ordinary words

Go back to the canteen. Three things can happen when a second bill arrives.

Answers	What the new clue does	Canteen version
Exactly one	It tells you something genuinely new. Between them the clues pin every unknown down.	Anjali's bill and Vikram's bill. Different mixes, so the prices get fixed.
None	It contradicts what the earlier clues already forced. No numbers can satisfy both.	One bill says 2 teas cost 20; another says 2 teas cost 30. Somebody typed it wrong.
Infinitely many	It repeats what you already knew. Nothing new is pinned down, so choices survive.	The second bill is just the first one doubled: 4 teas and 6 samosas for 130. It tells you nothing new.

That is the entire theory. A new clue can *contradict*, *repeat*, or *add something new* — and those three possibilities are exactly the three fates.

▷ Say it to yourself like this

Every clue you add either fights what you already know (no answers), says nothing you did not already know (endless answers), or narrows things down further (heading towards one answer).

The same three cases with real equations

Now watch it happen. Start with two clues and hold them fixed:

$$x_1 + x_2 + x_3 = 3 \quad (7)$$

$$x_1 - x_2 + 2x_3 = 2 \quad (8)$$

Add them together. The x_2 terms cancel ($+x_2$ and $-x_2$), and $3 + 2 = 5$:

$$2x_1 + 3x_3 = 5. \quad (9)$$

Read that carefully. We did not add any new information — we only combined what we already had. So (9) is something these two clues **force** to be true, whatever the answer turns out to be. Hold on to it: it decides everything below.

Case: no answers

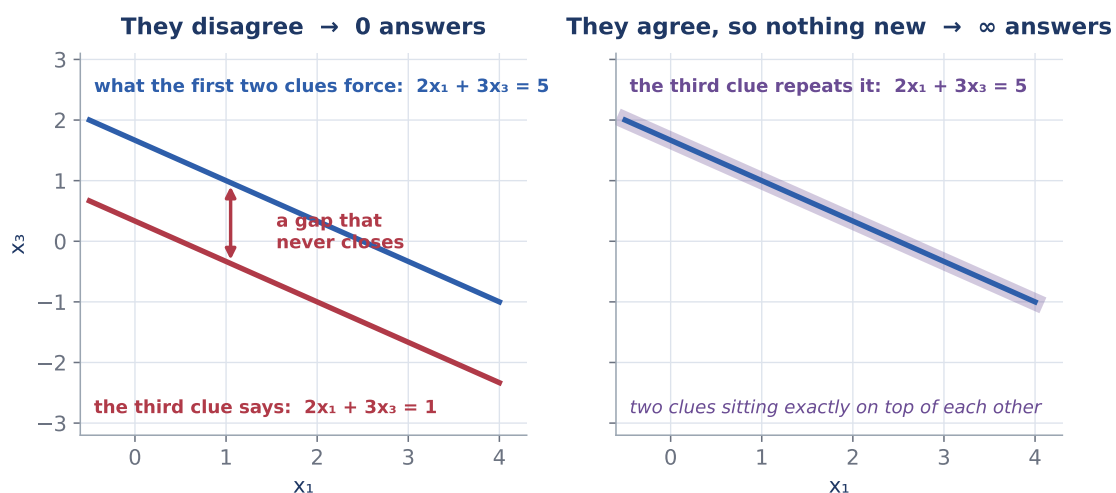
Let the third clue be $2x_1 + 3x_3 = 1$.

But (9) says that same quantity must be 5. One number cannot be both 5 and 1. The clues are fighting each other, so no set of values can satisfy them all. The system is called **inconsistent**, and it has **zero** solutions.

Case: infinitely many answers

Let the third clue be $2x_1 + 3x_3 = 5$ instead.

This is precisely what (9) already told us. The third clue is an echo — true, but useless. We are effectively still working with two clues for three unknowns, which is not enough to pin all three down. One unknown stays loose, and there are **infinitely many** solutions.



How to read it. Both panels plot the third clue against what the first two already force. Left: same tilt, different demand, so the lines never meet — no answers. Right: the third clue lies exactly on top of the forced fact — endless answers.

Case: exactly one answer

Let the third clue be $x_2 + x_3 = 2$. This one is genuinely new — it says something about x_2 , which (9) never mentioned. Now everything locks.

✓ Solving the system with a genuinely new third clue

The three clues are $x_1 + x_2 + x_3 = 3$, $x_1 - x_2 + 2x_3 = 2$, $x_2 + x_3 = 2$.

Step 1. Subtract the third clue from the first. $(x_1 + x_2 + x_3) - (x_2 + x_3) = 3 - 2$. The x_2 and x_3 terms cancel completely, leaving $x_1 = 1$.

Step 2. Put $x_1 = 1$ into the second clue. $1 - x_2 + 2x_3 = 2$, so $-x_2 + 2x_3 = 1$.

Step 3. Add the third clue to that. $(-x_2 + 2x_3) + (x_2 + x_3) = 1 + 2$. The x_2 terms cancel: $3x_3 = 3$, so $x_3 = 1$.

Step 4. Back into the third clue. $x_2 + 1 = 2$, so $x_2 = 1$.

Step 5. Check all three original clues. $1 + 1 + 1 = 3$ ✓ $1 - 1 + 2 = 2$ ✓ $1 + 1 = 2$ ✓

The unique answer is $(1, 1, 1)$.

When answers are endless, you can still write them all down

Return to the infinite case: clues (7), (8), and the echo $2x_1 + 3x_3 = 5$. “Infinitely many answers” sounds like the end of the conversation. It is not — you can describe every single one of them in two short lines.

The trick: let one unknown be *whatever you like*, then watch the others fall into place.

Subtract (8) from (7): $2x_2 - x_3 = 1$, so $x_2 = \frac{1}{2} + \frac{1}{2}x_3$. And rearranging (9) gives $x_1 = \frac{5}{2} - \frac{3}{2}x_3$. Together:

$$x_1 = \frac{5}{2} - \frac{3}{2}x_3, \quad x_2 = \frac{1}{2} + \frac{1}{2}x_3, \quad x_3 = \text{anything you like.} \quad (10)$$

x_3 is called a **free variable** — you choose it. x_1 and x_2 are **pivot variables** (also called *basic variables*) — once x_3 is chosen, they have no choice left.

★ Ordering a thali

The restaurant offers a fixed thali with one choice: pick your sweet. Once you have picked, the rest of the plate is decided — that dal, that sabzi, that roti count. Endlessly many thalis are available (one per sweet), but each is completely determined the instant you choose. The sweet is the free variable.

✓ Turning the free choice into actual answers

Use (10). Every choice of x_3 produces a genuine solution.

Choice 1: let $x_3 = 0$. $x_1 = \frac{5}{2} - 0 = 2.5$, $x_2 = \frac{1}{2} + 0 = 0.5$. Answer: $(2.5, 0.5, 0)$.

Check: $2.5 + 0.5 + 0 = 3$ ✓ $2.5 - 0.5 + 0 = 2$ ✓ $2(2.5) + 3(0) = 5$ ✓

Choice 2: let $x_3 = 1$. $x_1 = \frac{5}{2} - \frac{3}{2} = 1$, $x_2 = \frac{1}{2} + \frac{1}{2} = 1$. Answer: $(1, 1, 1)$.

Check: $1 + 1 + 1 = 3$ ✓ $1 - 1 + 2 = 2$ ✓ $2(1) + 3(1) = 5$ ✓

Choice 3: let $x_3 = 3$. $x_1 = \frac{5}{2} - \frac{9}{2} = -2$, $x_2 = \frac{1}{2} + \frac{3}{2} = 2$. Answer: $(-2, 2, 3)$.

Check: $-2 + 2 + 3 = 3 \checkmark$ $-2 - 2 + 6 = 2 \checkmark$ $-4 + 9 = 5 \checkmark$

Conclusion. Three choices, three valid answers, and there are as many more as there are numbers. Yet two short formulas describe all of them. “Infinitely many” does not mean “unknowable”.

Part IV shows how to find the free variables automatically, without staring at the equations and hoping.

✗ **Never say “two solutions”**

If you ever finish a problem believing a linear system has exactly two answers, you have made an arithmetic mistake. Go back and check. The count is 0, 1, or infinity — there is nothing in between.

• **Remember this**

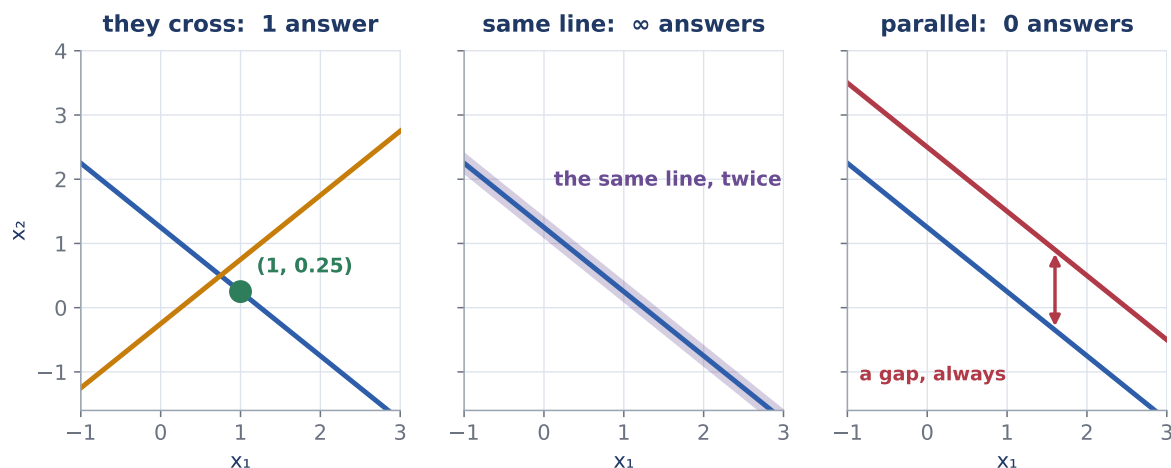
Zero, one, or infinitely many. Each new clue either contradicts what you already know (zero), repeats it (infinitely many), or genuinely narrows things down (heading for one). When answers are endless, free variables let you write every one of them in a couple of lines.

14 Seeing it: lines that cross, walls that meet

Pictures make the three fates obvious, so it is worth building the picture properly.

Two unknowns: every clue is a line

With two unknowns, a clue like $ax_1 + bx_2 = c$ is satisfied by a whole line of points. A solution of the *system* must sit on every line at once, so the answer is wherever the lines meet. Three arrangements are possible, and they are exactly the three fates.



How to read it. Left: two lines with different tilts must cross somewhere, and they cross once — one answer. Middle: the two clues describe the same line, so every point on it works — endless answers. Right: same tilt, different position, so they never meet — no answers.

✓ Finding a crossing point by hand

Where do $4x_1 + 4x_2 = 5$ and $2x_1 - 4x_2 = 1$ cross?

Step 1. Add the two clues. The x_2 terms are $+4x_2$ and $-4x_2$, so they vanish: $6x_1 = 6$, giving $x_1 = 1$.

Step 2. Put $x_1 = 1$ into the first clue. $4 + 4x_2 = 5$, so $4x_2 = 1$ and $x_2 = 0.25$.

Step 3. Check both. Clue 1: $4(1) + 4(0.25) = 4 + 1 = 5$ ✓ Clue 2: $2(1) - 4(0.25) = 2 - 1 = 1$ ✓

The lines cross at $(1, 0.25)$ — the only point loyal to both clues.

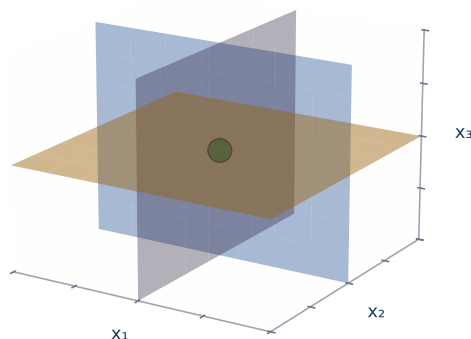
Three unknowns: every clue is a flat wall

With three unknowns, one clue $ax_1 + bx_2 + cx_3 = d$ is satisfied by a flat sheet of points floating in space — a **plane**. Picture a wall, or a sheet of glass.

Two walls that are not parallel meet along a straight line, the way a wall meets the floor. So after two clues, the surviving candidates form a line. The third clue then decides everything:

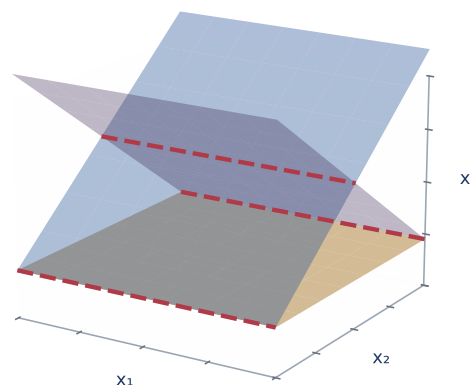
- It **cuts** that line at a single point $\Rightarrow$ one answer.
- It **contains** that line, like two pages sharing a spine $\Rightarrow$ endless answers.
- It **misses** the line entirely $\Rightarrow$ no answers.

three walls, one shared point: 1 answer



the green dot obeys all three clues

the prism trap: 0 answers



red creases: each pair meets, but the creases never touch

How to read it. Left: three walls all passing through one green point — the unique solution. Right: the “prism” trap. Each *pair* of walls meets along a red crease, but the three creases are parallel, so no single point lies on all three walls.

✗ Agreeing in pairs is not agreeing as a group

The prism on the right is the classic trap. Wall 1 meets wall 2, wall 2 meets wall 3, wall 1 meets wall 3 — every pair gets along fine. And yet there is no point common to all three. Checking your clues two at a time is not enough; they must agree all together.

More than three unknowns

With four or more unknowns you cannot draw the picture, and that is fine. Each clue still carves out a flat slab of the space — now called a **hyperplane** — and the solutions are still whatever survives all the slabs at once. The three fates do not change, in any number of dimensions.

ML This is the space your model lives in

A model with three weights lives in exactly the picture above: each training example is one wall in weight-space, and the weights that explain all your data sit where the walls meet. Real models have millions of weights and millions of walls, so nobody draws it — but the mental image of “walls meeting” is accurate, and it explains a lot. When people say a model is *over-parameterised*, they mean there are far more unknowns than walls, so the walls meet in a huge shared region rather than a single point — endless answers, and the training procedure picks one of them.

• Remember this

Two unknowns give lines, three give walls, more give hyperplanes. In every case the solutions are what all the clues share: a point (one answer), a line or bigger (endless answers), or nothing at all (no answers).

Part III · Matrix arithmetic

Adding, scaling, multiplying, flipping and undoing grids of numbers

15 Adding and scaling matrices

Good news: you already know this. A matrix is a grid, and the rules are the vector rules applied to every cell.

▷ Cell by cell, nothing clever

To add two matrices, add the numbers that sit in the same position. To scale a matrix, multiply every number in it by the same scalar. Cells never mix with their neighbours.

In symbols:

$$(A + B)_{ij} = a_{ij} + b_{ij}, \quad (\lambda A)_{ij} = \lambda a_{ij}. \quad (11)$$

Addition only works between matrices of the *same shape* — a 3×2 can only be added to another 3×2 , for the same reason you cannot add a list of three numbers to a list of four. Scaling works on any shape, and never changes the shape.

★ Two months of sales

You have last month's sales table (shops down the side, products across the top) and this month's, in exactly the same layout. Want the two-month total? Add the cells that sit in the same place. Want to forecast a month that is 10% busier? Multiply every cell by 1.1. Nobody would do either differently.

✓ One sum and one scale, in full

Let $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ and $B = \begin{bmatrix} 5 & 0 \\ -1 & 2 \end{bmatrix}$.

Step 1. Check the shapes. Both are 2×2 . Addition is allowed.

Step 2. Top row. $1 + 5 = 6$ and $2 + 0 = 2$.

Step 3. Bottom row. $3 + (-1) = 2$ and $4 + 2 = 6$.

Step 4. Collect. $A + B = \begin{bmatrix} 6 & 2 \\ 2 & 6 \end{bmatrix}$.

Step 5. Now scale. $3A$ multiplies every entry by 3: $3A = \begin{bmatrix} 3 & 6 \\ 9 & 12 \end{bmatrix}$.

Step 6. Sense-check. The shape did not change, and no cell borrowed anything from a neighbour. Adding and scaling are strictly local operations.

Scaling behaves the way you would hope, which is worth stating once so you can stop worrying about it. For any numbers λ, ψ (lambda and psi) and matrices A, B of matching shape:

$$(\lambda\psi)A = \lambda(\psi A), \quad (\lambda + \psi)A = \lambda A + \psi A, \quad \lambda(A + B) = \lambda A + \lambda B.$$

ML Every training step is one scale and one add

When a model learns, it repeatedly nudges its weights:

$$W_{\text{new}} = W_{\text{old}} + (-\eta) G.$$

Here G is a matrix of the same shape as W saying which direction each weight should move to reduce the error, and η (eta, the *learning rate*) is a small number like 0.01 saying how big a step to take. Scale G by $-\eta$, add it to W , repeat a few million times. That is gradient descent, and structurally it is nothing more than this section.

• Remember this

Matrix addition and scaling are cell-by-cell. Shapes must match to add; scaling never changes shape. There is nothing hidden here.

16 Multiplying: first a matrix times a vector

Matrix multiplication is the one operation that is *not* cell by cell, and it is the most important operation in all of machine learning. We build it in two stages: first a matrix times a vector (which you have already done, in a shop), then a matrix times a matrix (which is just the first thing repeated).

Stage 0: the dot product, which is just a bill

★ Reading a bill

You buy 2 kg of rice and 3 kg of dal. Rice costs Rs. 50 per kg, dal Rs. 20 per kg. What do you pay?

$$2 \times 50 + 3 \times 20 = 100 + 60 = \mathbf{160}.$$

Pair each quantity with its own price, multiply the pairs, add the results. Everyone on earth computes a bill this way.

That operation — pair up, multiply, add — has a name. For two vectors of the same length, the **dot product** is

$$\mathbf{u} \cdot \mathbf{v} = u_1 v_1 + u_2 v_2 + \cdots + u_n v_n.$$

It takes two lists and returns *one number*. Quantities in, prices in, total out.

Stage 1: a matrix times a vector is several bills at once

Now suppose you have three shops, each buying the same two goods in different quantities. Put the quantities in a matrix (one shop per row) and the prices in a vector. Each shop's bill is one dot product: its row against the price list. Three rows, three bills.

That is exactly what $A\mathbf{x}$ means.

▷ Row meets list

To compute $A\mathbf{x}$: take row 1 of A , dot it with $\mathbf{x}$, and that is the first number of the answer. Take row 2, dot it with $\mathbf{x}$, and that is the second. Keep going. A matrix with m rows gives an answer with m numbers.

One row of the grid meets the whole list — that is one output number

$$\begin{bmatrix} 2 & 3 & 5 \\ 4 & -2 & -7 \\ 9 & 5 & -3 \end{bmatrix} \times \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 13 \\ -7 \\ 16 \end{bmatrix}$$

row 2 walked against the list: $4 \times 1 + (-2) \times 2 + (-7) \times 1 = 4 - 4 - 7 = -7$

Repeat for row 1 and row 3. Three rows in, three numbers out.

How to read it. The highlighted row of the grid walks along the highlighted list, multiplying pair by pair and adding. That produces exactly one number in the answer. Repeat for every row.

✓ A 3×3 matrix times a vector, every step shown

Compute Ax for $A = \begin{bmatrix} 2 & 3 & 5 \\ 4 & -2 & -7 \\ 9 & 5 & -3 \end{bmatrix}$ and $x = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$.

Step 1. Check the shapes. A has 3 columns and x has 3 entries. They match, so the multiplication is legal. The answer will have 3 entries, one per row of A .

Step 2. Row 1 against x . $2(1) + 3(2) + 5(1) = 2 + 6 + 5 = 13$.

Step 3. Row 2 against x . $4(1) + (-2)(2) + (-7)(1) = 4 - 4 - 7 = -7$.

Step 4. Row 3 against x . $9(1) + 5(2) + (-3)(1) = 9 + 10 - 3 = 16$.

Step 5. Collect. $Ax = (13, -7, 16)$.

Step 6. Sense-check. Three rows in, three numbers out. Each output number used the whole of x and exactly one row of A .

The second reading: mixing the columns

There is another way to compute the very same thing, and it will matter enormously in Part IV. Instead of walking rows, take each number in x as an instruction about *how much of each column* of A to take:

$$\begin{bmatrix} 2 & 3 & 5 \\ 4 & -2 & -7 \\ 9 & 5 & -3 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} = 1 \begin{bmatrix} 2 \\ 4 \\ 9 \end{bmatrix} + 2 \begin{bmatrix} 3 \\ -2 \\ 5 \end{bmatrix} + 1 \begin{bmatrix} 5 \\ -7 \\ -3 \end{bmatrix} = \begin{bmatrix} 2 + 6 + 5 \\ 4 - 4 - 7 \\ 9 + 10 - 3 \end{bmatrix} = \begin{bmatrix} 13 \\ -7 \\ 16 \end{bmatrix}.$$

Same answer, different story. The row reading says “each row is a bill”. The column reading says “ Ax is a linear combination of the columns of A , and x holds the recipe”. Both are true, always. Keep both.

• Remember this

Ax can be read two ways: each row of A dotted with x , or a mix of the columns of A with x as the recipe. The second reading is the one that unlocks Part IV.

17 Multiplying a matrix by a matrix

Once you can multiply a matrix by a vector, matrices are easy: a matrix on the right is just several vectors standing side by side, so do the vector job once per column.

For $A \in \mathbb{R}^{m \times k}$ and $B \in \mathbb{R}^{k \times n}$, the product $C = AB$ is $m \times n$, and the entry in row i , column j is row i of A dotted with column j of B :

$$c_{ij} = \sum_{l=1}^k a_{il} b_{lj} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{ik}b_{kj}. \quad (12)$$

The letter l is just a counter for the walk, from 1 to k . Do not let the $\sum$ sign worry you — it says “add up all of these”, and the expanded version to its right is the same statement in longhand.

The shape rule, and why it exists

$$(m \times \underline{k}) \times (\underline{k} \times n) = (m \times n)$$

The two underlined numbers must be equal. That is not an arbitrary rule: row i of A has k numbers in it, and column j of B must have k numbers too, or there is nothing to pair them with. The outer two numbers survive and give the shape of the answer.

✗ AB existing tells you nothing about BA

If A is 2×3 and B is 3×4 , then AB is a perfectly good 2×4 matrix — and BA does not exist at all, because 4 and 2 do not match. Even when both exist, they are usually different matrices. Order is part of the instruction, not a detail.

✓ Every cell of a 2×2 product — and the order trap

Let $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$.

Step 1. Top-left, c_{11} . Row 1 of A meets column 1 of B : $1 \times 5 + 2 \times 7 = 5 + 14 = 19$.

Step 2. Top-right, c_{12} . Row 1 meets column 2: $1 \times 6 + 2 \times 8 = 6 + 16 = 22$.

Step 3. Bottom-left, c_{21} . Row 2 meets column 1: $3 \times 5 + 4 \times 7 = 15 + 28 = 43$.

Step 4. Bottom-right, c_{22} . Row 2 meets column 2: $3 \times 6 + 4 \times 8 = 18 + 32 = 50$.

So $AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$.

Step 5. Now swap the order. The same walks, but with B 's rows and A 's columns:

$$BA = \begin{bmatrix} 5(1) + 6(3) & 5(2) + 6(4) \\ 7(1) + 8(3) & 7(2) + 8(4) \end{bmatrix} = \begin{bmatrix} 23 & 34 \\ 31 & 46 \end{bmatrix}.$$

Step 6. Compare. $AB \neq BA$, even though both exist and both are 2×2 . Matrix multiplication does not care about your instincts from ordinary arithmetic.

Rules that do hold

Three familiar rules survive, and they are used constantly:

- **Grouping is free** (associativity): $(AB)C = A(BC)$. You may bracket a long product wherever you like. This matters in practice — one bracketing can be dramatically cheaper to compute than another.

- **Multiplication spreads over addition** (distributivity): $(A + B)C = AC + BC$ and $A(C + D) = AC + AD$.
- **There is a do-nothing matrix** (the identity): $I_m A = A I_n = A$.

The **identity matrix** I is square, with 1s down the main diagonal and 0s everywhere else:

$$I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

It plays exactly the role the number 1 plays in ordinary arithmetic: multiplying by it changes nothing. It is worth checking once that this really works, so multiply $I_2 \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$: row 1 of I is $(1, 0)$, and dotting it with the columns $(1, 3)$ and $(2, 4)$ gives 1 and 2 — the original first row, untouched. The 1 picks out one entry and the 0 discards the rest.

× The one rule that breaks

$AB \neq BA$ in general. This is the single biggest difference between matrix arithmetic and the arithmetic you grew up with, and it is the source of a large share of all bugs in numerical code. When you rearrange an expression, you may regroup brackets freely — but you may never quietly swap two factors.

ML One layer of a neural network is one matrix multiply

Feed 32 images, each flattened to 784 numbers, into a layer with 128 units. The data is a 32×784 matrix; the layer's weights are a 784×128 matrix. Multiply them: the 784s meet and cancel, leaving a 32×128 result — 32 images, 128 numbers each. Then a small nonlinear function is applied and the next layer repeats the trick.

That is the entire forward pass of a deep network: matrix multiply, small function, matrix multiply, small function. Graphics cards exist, commercially, to do that first step astonishingly fast.

18 The transpose: flipping a grid

The **transpose** of A , written A^T , is A turned over along its main diagonal: rows become columns and columns become rows. In symbols, the entry at (i, j) of A^T is the entry at (j, i) of A .

★ Turning a table sideways

Your sales table has shops down the side and products across the top. Your colleague prefers products down the side and shops across the top. Same numbers, same information, rotated layout. That rotation is the transpose — and every spreadsheet program has a button for it.

✓ Transposing, and what happens to the shape

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \implies A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}.$$

Step 1. Row 1 of A is $(1, 2, 3)$; it becomes column 1 of A^T .

Step 2. Row 2 of A is $(4, 5, 6)$; it becomes column 2 of A^T .

Step 3. Shape. A was 2×3 ; A^T is 3×2 . Transposing always swaps the two shape numbers.

Step 4. Spot-check one entry. $a_{12} = 2$, and sure enough the entry at row 2, column 1 of A^T is also 2. Row and column indices trade places.

Three rules, all easy to believe:

$$(A^T)^T = A, \quad (A + B)^T = A^T + B^T, \quad (AB)^T = B^T A^T.$$

The first two are unsurprising — flip twice and you are back; flipping a sum flips each piece. The third one is the interesting one: flipping a *product* also **reverses the order**.

★ **Socks and shoes**

To get dressed you put socks on, then shoes. To undo it you take the shoes off first, then the socks. Undo-style operations always come off in reverse order. The transpose of a product behaves the same way.

✓ **Checking $(AB)^T = B^T A^T$ with real numbers**

Reuse $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ and $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$, for which we found $AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$.

Step 1. Flip the product. $(AB)^T = \begin{bmatrix} 19 & 43 \\ 22 & 50 \end{bmatrix}$.

Step 2. Flip each factor. $B^T = \begin{bmatrix} 5 & 7 \\ 6 & 8 \end{bmatrix}$, $A^T = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$.

Step 3. Multiply in the reversed order.

$$B^T A^T = \begin{bmatrix} 5(1) + 7(2) & 5(3) + 7(4) \\ 6(1) + 8(2) & 6(3) + 8(4) \end{bmatrix} = \begin{bmatrix} 19 & 43 \\ 22 & 50 \end{bmatrix}.$$

It matches Step 1 exactly. ✓

Step 4. Now try the wrong order, to see it fail.

$$A^T B^T = \begin{bmatrix} 1(5) + 3(6) & 1(7) + 3(8) \\ 2(5) + 4(6) & 2(7) + 4(8) \end{bmatrix} = \begin{bmatrix} 23 & 31 \\ 34 & 46 \end{bmatrix} \neq (AB)^T.$$

Step 5. Moral. The reversal is not decoration. Get it wrong and you get a different matrix.

ML Where you will actually see A^T

The transpose is everywhere once you start reading model equations. A dot product between two column vectors is written $\mathbf{u}^T \mathbf{v}$. The attention step inside a transformer computes QK^T . And when a network learns, the error signal is pushed backwards through a layer by multiplying with W^T — the same weights, read the other way round. You do not need those details yet; you only need to not flinch when a superscript T appears.

19 The inverse: an undo button, and when it exists

★ Undoing

Multiply a number by 5 and you can undo it by dividing by 5. Multiply by 0.2 and you undo it by dividing by 0.2. But multiply by 0 and you are stuck — everything collapses to zero and no division brings it back, because the information is gone.

Matrices behave the same way. Most of them have an undo. Some of them destroy information, and those have none.

For a square matrix A , the **inverse** A^{-1} is the matrix that undoes it:

$$AA^{-1} = A^{-1}A = I.$$

Do A , then do A^{-1} , and you are back where you started.

Working out the 2×2 case

Does every square matrix have one? The smallest interesting case answers it. Take a general 2×2 matrix and a cleverly chosen partner — swap the diagonal entries, flip the signs of the other two:

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}, \quad B = \begin{bmatrix} a_{22} & -a_{12} \\ -a_{21} & a_{11} \end{bmatrix}.$$

Multiply them out with the row-meets-column rule. The off-diagonal cells cancel to zero (for instance, top-right is $a_{11}(-a_{12}) + a_{12}(a_{11}) = 0$), and both diagonal cells come out as the same number:

$$AB = (a_{11}a_{22} - a_{12}a_{21}) I_2. \quad (13)$$

So B almost undoes A — it lands on a multiple of the identity rather than on the identity itself. Divide B by that multiple and you have the real thing:

$$A^{-1} = \frac{1}{a_{11}a_{22} - a_{12}a_{21}} \begin{bmatrix} a_{22} & -a_{12} \\ -a_{21} & a_{11} \end{bmatrix}, \quad (14)$$

which works *only when* $a_{11}a_{22} - a_{12}a_{21} \neq 0$, because dividing by zero is not allowed.

That gatekeeper number has a name: the **determinant**, written $\det A$. And it settles the question completely:

▷ The rule for inverses

A square matrix has an inverse exactly when its determinant is not zero. A square matrix with $\det A = 0$ is called **singular** and has no inverse at all.

✓ Building a real undo button

Let $A = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$.

Step 1. The determinant. $\det A = a_{11}a_{22} - a_{12}a_{21} = 2(1) - 1(1) = 1$. Not zero, so an inverse exists.

Step 2. Build it. Swap the diagonal (2 and 1 trade places), negate the other two, divide by 1:

$$A^{-1} = \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix}.$$

Step 3. Verify AA^{-1} , cell by cell.

Top-left: $2(1) + 1(-1) = 1$. Top-right: $2(-1) + 1(2) = 0$.

Bottom-left: $1(1) + 1(-1) = 0$. Bottom-right: $1(-1) + 1(2) = 1$.

Step 4. Read the result. $AA^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I$. The button really does undo.

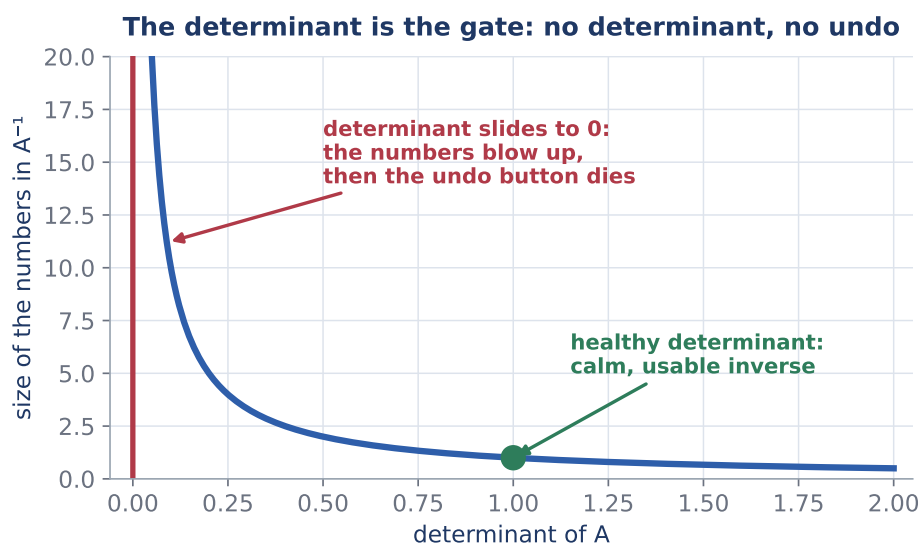
✓ A matrix with no undo, and why

Let $S = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$.

Step 1. The determinant. $\det S = 1(4) - 2(2) = 4 - 4 = 0$. No inverse.

Step 2. Look at why. Row 2 is exactly twice row 1. As a pair of clues, they say “ $x_1 + 2x_2 =$ something” twice over — the second clue is an echo, in the exact sense of Part II. The matrix carries less information than its size suggests.

Step 3. Connect it to the three fates. A system built on S can never have exactly one answer. Depending on the right-hand side it will have none (the two clues contradict) or endlessly many (they agree). A zero determinant is the algebra telling you the same thing the geometry told you.



How to read it. Every entry of A^{-1} carries a factor of $1/\det A$. As the determinant drifts towards zero the entries grow without limit, and at exactly zero the inverse ceases to exist.

Two packaging rules to file away for later:

$$(AB)^{-1} = B^{-1}A^{-1} \quad (\text{socks and shoes again}), \quad (A+B)^{-1} \neq A^{-1} + B^{-1} \text{ in general.}$$

ML When the determinant goes small, your model gets nervous

Suppose your dataset has a column for “size in square feet” and another for “size in square metres”. They are the same information twice. The resulting matrix has a determinant of zero (or, with real measurement noise, very nearly zero), and the model cannot decide how to split the credit between the two columns — push one weight up and the other down and the predictions do not change at all.

Practitioners call this **multicollinearity**. The visible symptom is wildly unstable weights that swing about when you retrain on slightly different data. The cause is the figure above: a near-zero determinant makes the undo button explode.

• Remember this

The inverse undoes a matrix, and it exists exactly when the determinant is non-zero. A zero determinant means the rows repeat information — and that is the same phenomenon that gave us zero or infinitely many solutions in Part II.

Part IV · $A\mathbf{x} = \mathbf{b}$ and the one method that cracks it

Packing a system into three symbols, then solving any such system every time

20 The whole system in three symbols

Look back at the workshop. Four equations, three unknowns, twelve coefficients scattered across the page. All of that can be packed into three objects:

- A — the **coefficient matrix**: all the numbers multiplying the unknowns, arranged exactly as they appear. One row per equation.
- $\mathbf{x}$ — the **unknown vector**: the things you are solving for, stacked in a column.
- $\mathbf{b}$ — the **right-hand side**: the target numbers, stacked in a column.

And then the whole system is one short line: $A\mathbf{x} = \mathbf{b}$.

Here is the packing in action:

$$\left. \begin{array}{l} 2x_1 + 3x_2 + 5x_3 = 1 \\ 4x_1 - 2x_2 - 7x_3 = 8 \\ 9x_1 + 5x_2 - 3x_3 = 2 \end{array} \right\} \iff \begin{bmatrix} 2 & 3 & 5 \\ 4 & -2 & -7 \\ 9 & 5 & -3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 8 \\ 2 \end{bmatrix}. \quad (15)$$

Why does this work? Because of the row reading of $A\mathbf{x}$ from Part III. Row 1 of A dotted with $\mathbf{x}$ gives $2x_1 + 3x_2 + 5x_3$ — which is the left-hand side of the first equation. Setting that equal to the first entry of $\mathbf{b}$ reproduces the first equation exactly. Row i of $A\mathbf{x} = \mathbf{b}$ is equation i . Nothing has been lost or added; the equations have only been written more compactly.

× Check the shapes before you write anything

If A is $m \times n$, then $\mathbf{x}$ must have n entries (one per column of A , which is one per unknown) and $\mathbf{b}$ must have m entries (one per row, which is one per equation). Getting m and n the wrong way round is the most common slip in this whole subject.

21 The column view: solving is shopping for a mix

Now the idea the lecture is really building towards. Read $A\mathbf{x}$ by *columns* instead of rows, using the second reading from Part III, and the meaning of “solve” changes character completely.

Take this system — two equations, four unknowns:

$$\begin{bmatrix} 1 & 0 & 8 & -4 \\ 0 & 1 & 2 & 12 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 42 \\ 8 \end{bmatrix}. \quad (16)$$

Rewrite the multiplication so that each unknown scales a whole column:

$$x_1 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + x_2 \begin{bmatrix} 0 \\ 1 \end{bmatrix} + x_3 \begin{bmatrix} 8 \\ 2 \end{bmatrix} + x_4 \begin{bmatrix} -4 \\ 12 \end{bmatrix} = \begin{bmatrix} 42 \\ 8 \end{bmatrix}. \quad (17)$$

▷ Solving is finding the right mix of columns

Think of the columns of A as the moves you are allowed to make, and $\mathbf{b}$ as the place you must end up. Each unknown says how many times to take its move. Solving $A\mathbf{x} = \mathbf{b}$ means finding the mix of moves that lands you exactly on the target.

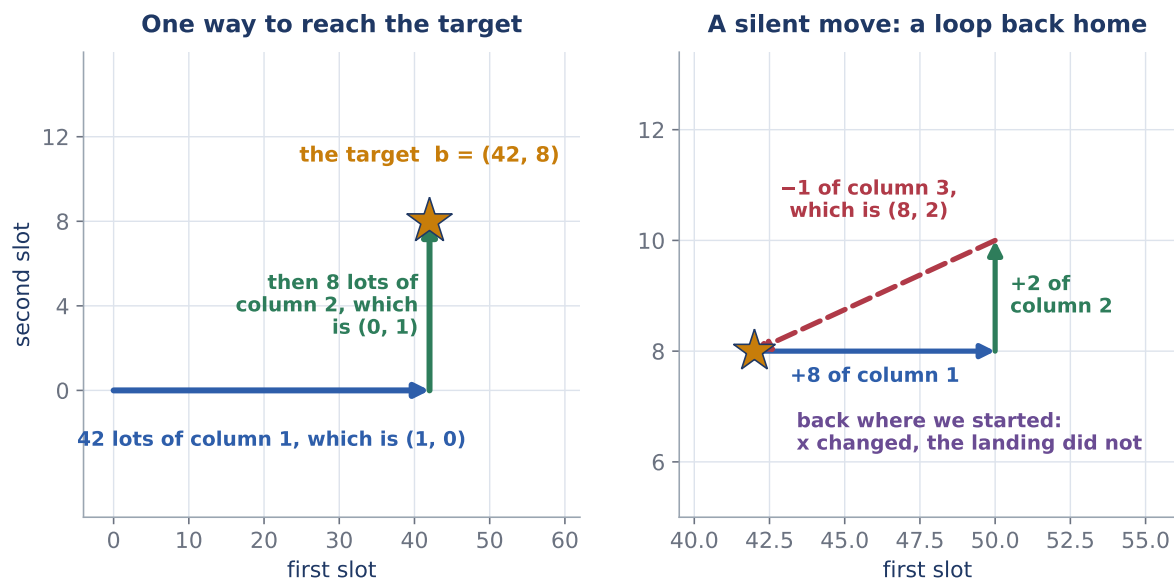
Or, in shopping language: the columns are the products on the shelf, $\mathbf{b}$ is the recipe you must reproduce, and $\mathbf{x}$ is your quantity for each product.

Finding one answer by looking

This particular matrix is kind to us. Its first two columns are $(1, 0)$ and $(0, 1)$ — a step purely along the first slot and a step purely along the second. To reach $(42, 8)$, take 42 of the first and 8 of the second and skip the other two entirely:

$$42 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + 8 \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 42 \\ 8 \end{bmatrix} \cdot \checkmark$$

So $\mathbf{x}_p = (42, 8, 0, 0)$ solves the system. It is called a **particular solution** — “particular” meaning *one specific answer*, with no claim to being the only one or the best one.



How to read it. Left: 42 steps of column one, then 8 steps of column two, landing on the target. Right: a loop that starts at the target, wanders off using columns one, two and three, and arrives back at the target — explained next.

ML This is how a layer of a network thinks

A linear layer’s output is a mix of its learned column vectors, and the input decides how much of each to use. “What combination of my building blocks reproduces this target?” is a question machine learning asks constantly — in regression, in recommender systems, in compression, in the first line of nearly every architecture.

22 Every answer at once: one landing plus the silent moves

Equation (16) has two clues and four unknowns. Two clues cannot possibly pin down four numbers, so by Part II there must be infinitely many solutions. We have found one. How do we describe all of them?

Step 1: a walk that goes nowhere

Look again at the columns:

$$\mathbf{c}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{c}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \mathbf{c}_3 = \begin{bmatrix} 8 \\ 2 \end{bmatrix}, \quad \mathbf{c}_4 = \begin{bmatrix} -4 \\ 12 \end{bmatrix}.$$

Notice something about $\mathbf{c}_3$. It equals $(8, 2)$ — but so does $8\mathbf{c}_1 + 2\mathbf{c}_2$. There are two different ways to get to the same place. So if you build it one way and *unbuild* it the other way, you have gone precisely nowhere:

$$8\mathbf{c}_1 + 2\mathbf{c}_2 - 1\mathbf{c}_3 + 0\mathbf{c}_4 = \mathbf{0} \quad \implies \quad \mathbf{s}_1 = (8, 2, -1, 0). \quad (18)$$

The same trick works on $\mathbf{c}_4$, which is $-4\mathbf{c}_1 + 12\mathbf{c}_2$:

$$-4\mathbf{c}_1 + 12\mathbf{c}_2 + 0\mathbf{c}_3 - 1\mathbf{c}_4 = \mathbf{0} \quad \implies \quad \mathbf{s}_2 = (-4, 12, 0, -1). \quad (19)$$

Call these **silent moves**. A silent move is a change to $\mathbf{x}$ that leaves $A\mathbf{x}$ completely untouched. In symbols, it solves $A\mathbf{s} = \mathbf{0}$. (The formal name for the collection of all silent moves is the *null space* of A , or its *kernel* — you will meet those words in a later session.)

★ A walk around the block

You leave your front door, walk around the block, and come back. Your position at the end is identical to your position at the start — but you certainly did something. A silent move is that walk: the recipe $\mathbf{x}$ genuinely changes, and the destination $A\mathbf{x}$ does not budge.

✓ Confirming the moves really are silent

A silent move must score zero on *every* row of A , where $A = \begin{bmatrix} 1 & 0 & 8 & -4 \\ 0 & 1 & 2 & 12 \end{bmatrix}$.

Step 1. $\mathbf{s}_1 = (8, 2, -1, 0)$, **row 1.** $1(8) + 0(2) + 8(-1) + (-4)(0) = 8 - 8 = \mathbf{0}$. ✓

Step 2. $\mathbf{s}_1$, **row 2.** $0(8) + 1(2) + 2(-1) + 12(0) = 2 - 2 = \mathbf{0}$. ✓

Step 3. $\mathbf{s}_2 = (-4, 12, 0, -1)$, **row 1.** $1(-4) + 0(12) + 8(0) + (-4)(-1) = -4 + 4 = \mathbf{0}$. ✓

Step 4. $\mathbf{s}_2$, **row 2.** $0(-4) + 1(12) + 2(0) + 12(-1) = 12 - 12 = \mathbf{0}$. ✓

Step 5. Verdict. Both moves change $\mathbf{x}$ and change $A\mathbf{x}$ by exactly nothing. Silent, as advertised.

Step 2: add silent moves to a known answer

Here is the payoff. If $\mathbf{x}_p$ solves the system and $\mathbf{s}$ is silent, then $\mathbf{x}_p + \mathbf{s}$ solves it too:

$$A(\mathbf{x}_p + \mathbf{s}) = A\mathbf{x}_p + A\mathbf{s} = \mathbf{b} + \mathbf{0} = \mathbf{b}.$$

Take any amounts λ_1 and λ_2 of the two silent moves and you still land on $\mathbf{b}$. So every solution of the system is captured by:

$$\mathbf{x} = \begin{bmatrix} 42 \\ 8 \\ 0 \\ 0 \end{bmatrix} + \lambda_1 \begin{bmatrix} 8 \\ 2 \\ -1 \\ 0 \end{bmatrix} + \lambda_2 \begin{bmatrix} -4 \\ 12 \\ 0 \\ -1 \end{bmatrix}, \quad \lambda_1, \lambda_2 \in \mathbb{R}. \quad (20)$$

This is the **general solution**: one specific landing, plus any mixture of the loops that go nowhere.

• The master recipe

Every solution = one particular solution + every possible mix of silent moves. Find one answer, find the loops, and you own all of them. This recipe never changes for the rest of the subject.

× Your answer may look different and still be right

We happened to pick $(42, 8, 0, 0)$ because it was easy to see, but $(50, 10, -1, 0)$ works just as well and would serve equally as the starting point. Different starting points, plus all the silent moves, describe the exact same infinite family. Do not be alarmed if your answer looks different from the printed one — check whether the difference is a silent move.

ML Why two training runs give different weights but the same predictions

Big networks have far more weights than they have constraints from the data, so they have an enormous supply of silent moves: whole directions in which you can change the weights without changing a single prediction. Train the same model twice with different random starting points and you land in different places in weight space — yet the two models behave identically. They differ by a silent move.

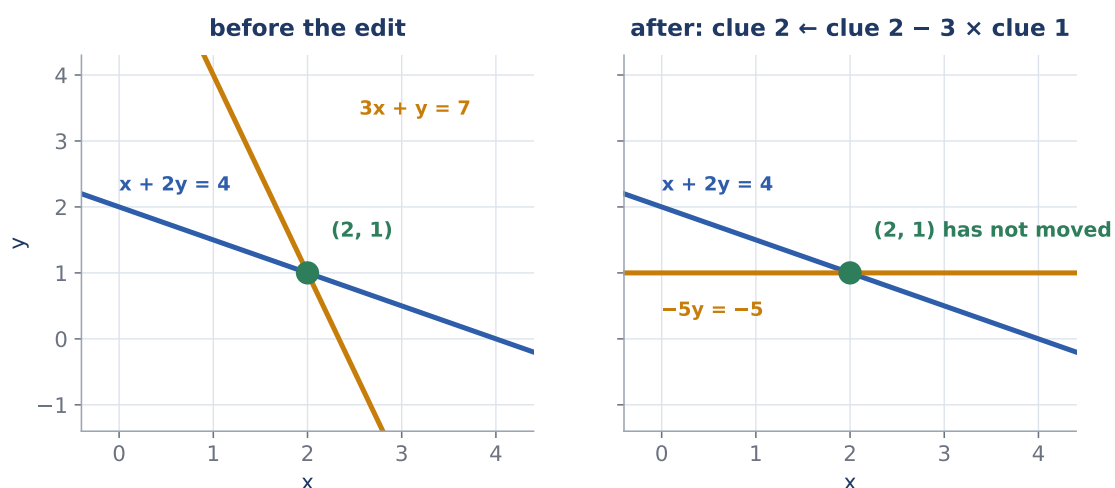
23 Three safe edits, and a shorthand for doing them

Real systems are not as friendly as (16). The strategy from here on: take a messy system and rewrite it as a tidy one *that has exactly the same answers*, then read the answers off the tidy one.

You saw the idea in the canteen: multiply a clue by 2, subtract another clue. Exactly three edits are safe, and they are called the **elementary row operations**:

1. **Swap two clues.** Reading the clues in a different order obviously cannot change what they mean.
2. **Multiply a clue by a non-zero number λ .** “ $x + y = 2$ ” and “ $2x + 2y = 4$ ” carry identical information — both sides scale together, so the truth of the statement is untouched.
3. **Add one clue to another (or a multiple of one to another).** If both clues are true, their sum is true as well. And you can always subtract it back again, so nothing is lost.

The common thread is **reversibility**. Every one of these edits can be undone, which is precisely why no solutions can be gained or lost along the way.



How to read it. The edit replaces the amber clue-line with a different line — but the new line still passes through the same crossing point. Row operations rotate the clues *around* the answer; the green dot never moves.

✗ **Why the multiplier must not be zero**

Multiply a clue by 0 and it becomes “ $0 = 0$ ” — true, but empty. You have erased a clue, and erasing cannot be undone. That is why the rule says $\lambda \neq 0$.

This is the same idea as $\det A = 0$ killing the inverse in Part III: operations that destroy information are not reversible, and irreversible operations are not safe.

The augmented matrix: stop writing the x ’s

Once you are only doing arithmetic on coefficients, writing $x_1, x_2, +$ and $=$ over and over is wasted effort. Strip them out and keep just the numbers, with a vertical bar standing in for the equals sign. That grid is the **augmented matrix**, written $[A \mid \mathbf{b}]$:

$$\begin{array}{rcl} x_1 + 2x_2 & = & 4 \\ 3x_1 + x_2 & = & 7 \end{array} \iff \left[\begin{array}{cc|c} 1 & 2 & 4 \\ 3 & 1 & 7 \end{array} \right]$$

The three safe edits now act on whole rows of this grid, the number after the bar included. That is the point: both sides of every equation move together, so the equations stay the same equations.

24 Gaussian elimination, from start to finish

▷ **The plan, in two sentences**

Sweep down using the three safe edits until the numbers form a staircase — zeros filling the bottom-left, each row starting further right than the one above it. Then **walk back up**, and each row hands you one more unknown.

The staircase is worth having a name for: the arrangement is called **row-echelon form**, where *echelon* simply means *steps*. In that shape, the bottom equation is tiny (often one unknown), the one above it adds one more, and so on — so reading it backwards is easy.

Here is a full worked sweep on the lecture’s own example. It carries a mystery number a on the right-hand side, which turns out to be a nice way of seeing how existence works.

$$\begin{array}{rcl} -2x_1 + 4x_2 - 2x_3 - x_4 + 4x_5 & = & -3 \\ 4x_1 - 8x_2 + 3x_3 - 3x_4 + x_5 & = & 2 \\ x_1 - 2x_2 + x_3 - x_4 + x_5 & = & 0 \\ x_1 - 2x_2 & - 3x_4 + 4x_5 & = a \end{array} \tag{21}$$

✓ **The full sweep down to the staircase**

Work on the augmented matrix. Every move below is one of the three safe edits.

Step 1. Write it out.

$$\left[\begin{array}{ccccc|c} -2 & 4 & -2 & -1 & 4 & -3 \\ 4 & -8 & 3 & -3 & 1 & 2 \\ 1 & -2 & 1 & -1 & 1 & 0 \\ 1 & -2 & 0 & -3 & 4 & a \end{array} \right]$$

Step 2. Swap $R_1 \leftrightarrow R_3$ to bring a clean 1 to the top-left. That leading 1 is the first **pivot** — the first non-zero entry of a row.

$$\left[\begin{array}{ccccc|c} 1 & -2 & 1 & -1 & 1 & 0 \\ 4 & -8 & 3 & -3 & 1 & 2 \\ -2 & 4 & -2 & -1 & 4 & -3 \\ 1 & -2 & 0 & -3 & 4 & a \end{array} \right]$$

Step 3. Clear everything below that pivot using the third safe edit, three times: $R_2 \leftarrow R_2 - 4R_1$, $R_3 \leftarrow R_3 + 2R_1$, $R_4 \leftarrow R_4 - R_1$.

$$\left[\begin{array}{ccccc|c} 1 & -2 & 1 & -1 & 1 & 0 \\ 0 & 0 & -1 & 1 & -3 & 2 \\ 0 & 0 & 0 & -3 & 6 & -3 \\ 0 & 0 & -1 & -2 & 3 & a \end{array} \right]$$

Step 4. Empty the last row: $R_4 \leftarrow R_4 - R_2 - R_3$. Every entry on the left cancels, and the right-hand side becomes $a - 2 - (-3) = a + 1$.

$$\left[\begin{array}{ccccc|c} 1 & -2 & 1 & -1 & 1 & 0 \\ 0 & 0 & -1 & 1 & -3 & 2 \\ 0 & 0 & 0 & -3 & 6 & -3 \\ 0 & 0 & 0 & 0 & 0 & a + 1 \end{array} \right]$$

Step 5. Tidy the pivots to 1: $R_2 \times (-1)$ and $R_3 \times (-\frac{1}{3})$. This is the second safe edit, and both multipliers are non-zero, so it is legal.

$$\left[\begin{array}{ccccc|c} 1 & -2 & 1 & -1 & 1 & 0 \\ 0 & 0 & 1 & -1 & 3 & -2 \\ 0 & 0 & 0 & 1 & -2 & 1 \\ 0 & 0 & 0 & 0 & 0 & a + 1 \end{array} \right]$$

This is row-echelon form: the staircase is built.

Step 6. Read the last row. In words it says $0 = a + 1$.

- If $a \neq -1$, that is a flat contradiction — something is being asked of nothing. **No solutions.**
- If $a = -1$, it reads $0 = 0$: harmless, and solutions exist.

One glance at one row settles whether the system is solvable at all. That is what the sweep buys you.

each pivot sits strictly to the right of the one above

					answers
1	-2	1	-1	1	0
0	0	1	-1	3	-2
0	0	0	1	-2	1
0	0	0	0	0	0

everything below the blue line is zero

pivots: first non-zero of each row

free columns (2 and 5): no pivot, your choice

How to read it. Circled entries are the pivots, one per non-zero row, each strictly to the right of the one above — that is the staircase. Everything below the blue line is zero. Columns 2 and 5 (purple) contain no pivot; those unknowns will be free.

25 Walking back up: reading off the answers

Set $a = -1$ so solutions exist. Now walk back up the staircase.

The rule for a particular solution: **set every free unknown to zero**, then read the rows from the bottom up, each one giving you a single new unknown. Here columns 2 and 5 have no pivot, so x_2 and x_5 are free; set both to 0.

✓ Back-substitution, bottom to top

The staircase rows say $x_1 - 2x_2 + x_3 - x_4 + x_5 = 0$, $x_3 - x_4 + 3x_5 = -2$, $x_4 - 2x_5 = 1$.

Step 1. Bottom row. $x_4 - 2(0) = 1$, so $x_4 = 1$.

Step 2. Middle row. $x_3 - 1 + 3(0) = -2$, so $x_3 = -1$.

Step 3. Top row. $x_1 - 2(0) + (-1) - (1) + 0 = 0$, so $x_1 = 2$.

Step 4. Collect. $\mathbf{x}_p = (2, 0, -1, 1, 0)$.

Step 5. Check against the original four equations — not the swept ones, the originals, because that is what we actually want to solve.

$$\text{E1: } -2(2) + 4(0) - 2(-1) - 1 + 4(0) = -4 + 2 - 1 = -3 \checkmark$$

$$\text{E2: } 4(2) - 8(0) + 3(-1) - 3(1) + 0 = 8 - 3 - 3 = 2 \checkmark$$

$$\text{E3: } 2 - 0 + (-1) - 1 + 0 = 0 \checkmark$$

$$\text{E4: } 2 - 0 - 3(1) + 4(0) = -1 = a \checkmark$$

All four hold.

Now the silent moves, one per free unknown

Silent moves solve the same staircase with every right-hand side set to zero. There is a clean recipe: turn one free unknown up to 1, leave the other free unknowns at 0, and walk back up as before. Then swap roles.

✓ Getting both silent moves

Move 1: let $x_2 = 1$ and $x_5 = 0$ (all right-hand sides now zero).

Bottom row: $x_4 - 2(0) = 0$, so $x_4 = 0$. Middle row: $x_3 - 0 + 0 = 0$, so $x_3 = 0$.

Top row: $x_1 - 2(1) + 0 - 0 + 0 = 0$, so $x_1 = 2$.

$\Rightarrow \mathbf{s}_1 = (2, 1, 0, 0, 0)$.

Move 2: let $x_5 = 1$ and $x_2 = 0$.

Bottom row: $x_4 - 2(1) = 0$, so $x_4 = 2$. Middle row: $x_3 - 2 + 3(1) = 0$, so $x_3 = -1$.

Top row: $x_1 - 0 + (-1) - 2 + 1 = 0$, so $x_1 = 2$.

$\Rightarrow \mathbf{s}_2 = (2, 0, -1, 2, 1)$.

Check $\mathbf{s}_2$ on the original equations (it must score 0 on every one):

E1: $-2(2) + 4(0) - 2(-1) - 1(2) + 4(1) = -4 + 2 - 2 + 4 = 0 \checkmark$

E2: $4(2) - 8(0) + 3(-1) - 3(2) + 1(1) = 8 - 3 - 6 + 1 = 0 \checkmark$

E3: $2 - 0 + (-1) - 2 + 1 = 0 \checkmark$

E4: $2 - 0 - 3(2) + 4(1) = 2 - 6 + 4 = 0 \checkmark$

So the complete answer to the system, when $a = -1$, is

$$\mathbf{x} = \begin{bmatrix} 2 \\ 0 \\ -1 \\ 1 \\ 0 \end{bmatrix} + \lambda_1 \begin{bmatrix} 2 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} + \lambda_2 \begin{bmatrix} 2 \\ 0 \\ -1 \\ 2 \\ 1 \end{bmatrix}, \quad \lambda_1, \lambda_2 \in \mathbb{R}. \quad (22)$$

One particular landing, plus every mix of two silent moves. Exactly the master recipe.

× A check worth doing yourself

The lecture slides print the second silent move as $(2, 0, -2, 1, 1)$. Test it on the first equation: $-2(2) + 4(0) - 2(-2) - 1(1) + 4(1) = -4 + 4 - 1 + 4 = 3$, which is not 0 — so that vector is not silent. The corrected move $(2, 0, -1, 2, 1)$ scores 0 on all four equations, as verified above.

The lesson is not that slides contain typos. It is that *you never need to wonder*. A claimed solution can always be checked in a minute by substitution, and a claimed silent move by checking it scores zero. Never outsource your checking.

26 The staircase rules: REF, RREF, pivots and free variables

Time to state the staircase precisely. A matrix is in **row-echelon form** (REF) when all four of these hold:

1. Rows that are entirely zero sit at the bottom.
2. In every non-zero row, the first non-zero entry is that row's **pivot**.
3. Each pivot sits strictly to the right of the pivot in the row above — the staircase.
4. Everything below a pivot, in its column, is zero.

From that shape, two words fall out:

- An unknown whose column *contains* a pivot is a **pivot variable** (also called *basic*). The staircase locks it down.
- An unknown whose column contains *no* pivot is a **free variable**. You choose it, and everything else adjusts.

This is what we were doing by eye in Part II, now made mechanical. And the counting rule is worth memorising: **one free variable means one silent move**. Two free variables, two silent moves. That is where the λ 's in the general solution come from.

The deluxe version: RREF

Keep sweeping — clearing *above* each pivot as well as below — until:

- every pivot equals exactly 1, and
- each pivot is the *only* non-zero number in its column.

That is **reduced row-echelon form** (RREF). The pivot columns then look exactly like columns of the identity matrix, which makes solutions readable by eye.

✓ All the silent moves of a matrix already in RREF

Let $A = \begin{bmatrix} \mathbf{1} & 3 & 0 & 0 & 3 \\ 0 & 0 & \mathbf{1} & 0 & 9 \\ 0 & 0 & 0 & \mathbf{1} & -4 \end{bmatrix}$, with pivots in bold.

Step 1. Identify. Pivots own columns 1, 3 and 4. Columns 2 and 5 are free. Two free columns, so expect two silent moves.

Step 2. Why free columns are easy here. Because the pivot columns are $\mathbf{c}_1 = (1, 0, 0)$, $\mathbf{c}_3 = (0, 1, 0)$, $\mathbf{c}_4 = (0, 0, 1)$, any other column can be read off directly as a mix of them — its entries *are* the recipe.

Step 3. Free column 2. It reads $(3, 0, 0)$, which is $3\mathbf{c}_1$. Build it one way, unbuild the other: $3\mathbf{c}_1 - 1\mathbf{c}_2 = \mathbf{0}$, giving the silent move $(3, -1, 0, 0, 0)$.

Step 4. Free column 5. It reads $(3, 9, -4)$, which is $3\mathbf{c}_1 + 9\mathbf{c}_3 - 4\mathbf{c}_4$. Cancelling gives $3\mathbf{c}_1 + 9\mathbf{c}_3 - 4\mathbf{c}_4 - 1\mathbf{c}_5 = \mathbf{0}$, so the move is $(3, 0, 9, -4, -1)$.

Step 5. Verify the second move row by row.

Row 1: $1(3) + 3(0) + 0(9) + 0(-4) + 3(-1) = 3 - 3 = \mathbf{0}$ ✓

Row 2: $0 + 0 + 1(9) + 0 + 9(-1) = \mathbf{0}$ ✓

Row 3: $0 + 0 + 0 + 1(-4) + (-4)(-1) = -4 + 4 = \mathbf{0}$ ✓

Step 6. Write every solution of $A\mathbf{x} = \mathbf{0}$.

$$\mathbf{x} = \lambda_1(3, -1, 0, 0, 0) + \lambda_2(3, 0, 9, -4, -1), \quad \lambda_1, \lambda_2 \in \mathbb{R}.$$

ML Counting pivots is counting how much your data actually says

Pivots count the directions your clues genuinely pin down. Free columns count the directions they leave loose. Ten thousand rows of data with only three pivots means your dataset, despite its size, constrains only three things — the rest is repetition.

That count has a name you will meet in a later session: the **rank** of the matrix. For now, the intuition is enough: pivots are information, free columns are freedom.

• Remember this

Sweep down to the staircase and read the bottom row to learn whether solutions exist. Then walk back up: free variables at zero gives one particular solution, and turning each free variable up to 1 in turn gives one silent move each. That recipe solves every linear system there is.

27 Computing an inverse with the same machinery

Part III gave a formula for 2×2 inverses. For bigger matrices the formulas become horrible — but elimination does not care about size, so we use it instead.

The reasoning comes in three short steps:

1. Column i of A^{-1} is the solution of $Ax = e_i$, where e_i is column i of the identity (all zeros with a single 1 in slot i). This is just $AA^{-1} = I$ read one column at a time.
2. Each of those systems is solvable by sweeping $[A | e_i]$ until the left half becomes I — at which point the right half *is* that column of the answer.
3. So do not run n separate sweeps. Park all n targets on the right at once — the whole identity matrix — and sweep once.

$$[A | I] \xrightarrow{\text{sweep}} [I | A^{-1}]$$

★ A flight recorder

The right-hand half starts as I , meaning “nothing done yet”. Every row operation you perform on the left is performed on the right too, so the right half quietly records everything you did. By the time the left half has been turned into I , the right half holds the total record of the operations that achieved it — and “everything that turns A into I ” is precisely what A^{-1} means.

✓ Inverting $A = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$ by sweeping

Step 1. Set up $[A | I]$. $\left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{array} \right]$

Step 2. $R_1 \leftrightarrow R_2$, to put a 1 in the pivot position. $\left[\begin{array}{cc|cc} 1 & 1 & 0 & 1 \\ 2 & 1 & 1 & 0 \end{array} \right]$

Step 3. $R_2 \leftarrow R_2 - 2R_1$, clearing below the pivot. $\left[\begin{array}{cc|cc} 1 & 1 & 0 & 1 \\ 0 & -1 & 1 & -2 \end{array} \right]$

Step 4. $R_2 \times (-1)$, making the second pivot equal to 1. $\left[\begin{array}{cc|cc} 1 & 1 & 0 & 1 \\ 0 & 1 & -1 & 2 \end{array} \right]$

Step 5. $R_1 \leftarrow R_1 - R_2$, clearing *above* the pivot too — this is the step that takes us all the way to RREF. $\left[\begin{array}{cc|cc} 1 & 0 & 1 & -1 \\ 0 & 1 & -1 & 2 \end{array} \right]$

Step 6. Read it off. The left half is I , so $A^{-1} = \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix}$ — exactly what the 2×2 formula gave in Part III, now obtained by row operations alone.

Step 7. Verify. Row 1 of AA^{-1} : $2(1) + 1(-1) = 1$ and $2(-1) + 1(2) = 0$. Row 2: $1(1) + 1(-1) = 0$ and $1(-1) + 1(2) = 1$. That is I . ✓

× When the sweep stalls, there is no inverse

If at some point you need a pivot in a column and every candidate entry is zero, with no row left to swap in, the left half can never become I . That is the same matrix for which $\det A = 0$. The method does not merely fail — it *proves* no inverse exists.

ML “Solve, don’t invert”

Numerical libraries almost never build A^{-1} in practice. If you need x with $Ax = b$, they run this elimination directly on $[A | b]$: it is faster, and it accumulates far less rounding error than computing a full inverse and then multiplying by it. “Solve, don’t invert” is a working engineer’s proverb, and this section is the reason behind it.

28 Where all of this was going

Take stock. In one session you have gone from “a vector is a list of numbers” to a complete, mechanical method for solving any system of linear equations, along with a way to describe every solution when there are infinitely many.

The chain of ideas, end to end:

lists of numbers $\rightarrow$ grids of numbers $\rightarrow$ clues about unknowns $\rightarrow$ lines and walls $\rightarrow Ax = \mathbf{b} \rightarrow$
the staircase $\rightarrow$ every solution.

The one loose end, and why it is the start of everything

Back in Part II we left a promise. Real datasets hand you thousands of clues for a handful of unknowns: a thousand flats, three weights. Such a system is called **overdetermined**, and it almost always meets the first fate — **no exact solution**. No three weights can reproduce a thousand rents perfectly, because the world has details no straight-line rule can capture.

Demanding perfection is therefore the wrong question. The right question is:

Which $\mathbf{w}$ makes $X\mathbf{w}$ come closest to $\mathbf{y}$?

That question is **linear regression**, and answering it is the subject of the next session. Its classic closed-form answer is

$$\mathbf{w} = (X^T X)^{-1} X^T \mathbf{y},$$

and the point of showing it here is that you can already read every symbol in it:

- X is your data matrix — one row per record, one column per feature (Part I).
- X^T is the transpose, the same table read the other way round (Part III).
- $X^T X$ is a matrix product, computed row-meets-column (Part III).
- $(\cdot)^{-1}$ is the inverse, which exists exactly when the determinant is non-zero — and which becomes unstable when two of your features are near-copies of each other (Part III).
- And in practice nobody forms that inverse: they solve the system by elimination instead (Part IV).

Every single tool in this reader is a working part of that one formula. Nothing here was preparation for its own sake.

• The recipe to carry into every problem

Write the system as $[A \mid \mathbf{b}]$ and sweep it down to the staircase. If any row reads “0 = something non-zero”, there is no solution — stop. Otherwise: set the free variables to zero and walk back up for one particular solution, then turn each free variable up to 1 in turn for one silent move each. The full answer is that particular solution plus every mix of the silent moves. And check your answer in the original equations.

Reference · Glossary, symbols and a self-test

Everything named in this reader, in one place — plus ten questions with full answers

Glossary

vector	An ordered list of numbers. Drawable as an arrow when it has two or three entries; perfectly valid with three hundred.
scalar	An ordinary single number, used to scale a vector or matrix.
component / entry	One of the numbers inside a vector or matrix.
linear combination	Scale some vectors, then add the results.
closure	Adding or scaling never takes you outside the set you started in.
vector space	A set that is closed under adding and scaling.
matrix	A grid of numbers: m rows by n columns.
shape	The pair (rows, columns). Always said rows first.
row vector	A $1 \times n$ matrix. A column vector is $m \times 1$.
dot product	Pair up two equal-length lists, multiply pairwise, add. Gives one number.
identity matrix I	Ones on the main diagonal, zeros elsewhere. Does nothing when you multiply by it.
transpose A^T	The grid flipped along its diagonal; rows become columns.
determinant $\det A$	The gatekeeper number. For 2×2 , $a_{11}a_{22} - a_{12}a_{21}$.
singular	A square matrix with $\det A = 0$. It has no inverse.
inverse A^{-1}	The undo button: $AA^{-1} = A^{-1}A = I$. Exists only when $\det A \neq 0$.
linear equation	A clue in which every unknown is multiplied only by a known number, and the pieces are added.
system	Several clues that must all hold at the same time.
consistent	A system that has at least one solution. Inconsistent means it has none.
augmented matrix	$[A \mid \mathbf{b}]$: the coefficients, a bar, then the right-hand sides.
elementary row operations	The three safe edits: swap two rows, scale a row by a non-zero number, add a multiple of one row to another.
row-echelon form (REF)	The staircase: zero rows at the bottom, each pivot strictly right of the one above, zeros below every pivot.
reduced REF (RREF)	The deluxe staircase: every pivot is 1 and is the only non-zero entry in its column.
pivot	The first non-zero entry of a non-zero row in echelon form.
pivot (basic) variable	An unknown whose column contains a pivot. The system locks it down.
free variable	An unknown whose column has no pivot. You may choose its value.
Gaussian elimination	Sweep down to the staircase with safe edits, then walk back up.
particular solution	One specific answer to $A\mathbf{x} = \mathbf{b}$. Not unique.
silent move	A change to $\mathbf{x}$ that leaves $A\mathbf{x}$ unchanged; it solves $A\mathbf{x} = \mathbf{0}$. (Formally: an element of the <i>null space</i> .)
general solution	One particular solution plus every mix of silent moves.

overdetermined	More equations than unknowns. Usually has no exact solution — which is where linear regression begins.
-----------------------	--

Symbols at a glance

$\mathbb{R}^n$	All lists of n real numbers.
$\mathbb{R}^{m \times n}$	All grids with m rows and n columns.
a_{ij}	The entry in row i , column j — row first, always.
$\mathbf{v}, \mathbf{x}, \mathbf{b}$	Vectors (bold, lower case).
A, B, X	Matrices (capitals).
λ, ψ	Plain numbers used for scaling (lambda, psi).
η	The learning rate in gradient descent (eta).
I_n	The $n \times n$ identity: the do-nothing matrix.
A^{-1}	The inverse of A : the undo button.
A^T	The transpose of A : the flip.
$\det A$	The determinant: the gatekeeper for invertibility.
$A\mathbf{x} = \mathbf{b}$	A whole system written in three symbols.
$[A \mid \mathbf{b}]$	The augmented matrix: coefficients, bar, right-hand sides.
$\mathbf{e}_i$	Column i of the identity: all zeros with a single 1 in slot i .
$\mathbf{0}$	The zero vector: every entry 0.
$\mathbf{x}_p$	A particular solution.
$X, \mathbf{y}, \mathbf{w}$	In machine learning: the data matrix, the targets, the weights.
$\sum_{l=1}^k$	“Add up all of these, for l running from 1 to k .”

Check yourself

Cover the answers. Work each one on paper. Anything you cannot do points at exactly one section to reread.

Q1. Is $(3, 2)$ the same vector as $(2, 3)$? Explain in one sentence.

Q2. Compute $(1, 4, -2) + (5, 0, 3)$, and then $-2 \times (3, -1, 0)$.

Q3. Is $3x_1 + x_1x_2 = 7$ a linear equation? If not, which part breaks the rule?

Q4. A is 4×2 and B is 2×5 . What shape is AB ? Does BA exist?

Q5. Find $\det A$ and A^{-1} for $A = \begin{bmatrix} 3 & 1 \\ 2 & 4 \end{bmatrix}$.

Q6. A system has 3 equations and 5 unknowns. Can it have exactly one solution?

Q7. How many solutions has $\{x + y = 5, 2x + 2y = 11\}$? Why?

Q8. Compute $A\mathbf{x}$ for $A = \begin{bmatrix} 1 & 2 \\ 0 & 3 \end{bmatrix}$ and $\mathbf{x} = (4, 1)$ — once by rows, once by mixing columns.

Q9. $\mathbf{x}_p = (1, 2, 0)$ solves $A\mathbf{x} = \mathbf{b}$, and $\mathbf{s} = (0, 1, -1)$ is a silent move. Write down two more solutions.

Q10. Three pens and two notebooks cost Rs. 130. One pen and four notebooks cost Rs. 160. Find both prices, and check your answer.

Answers

- A1.** No. A vector is an *ordered* list, so slot 1 holding 3 is a different statement from slot 1 holding 2 — just as house 32 is not house 23.
- A2.** $(1 + 5, 4 + 0, -2 + 3) = (6, 4, 1)$. And $-2 \times (3, -1, 0) = (-6, 2, 0)$: every entry doubled and flipped in sign.
- A3.** Not linear. The term x_1x_2 multiplies one unknown by another unknown, and the rule allows an unknown to be multiplied only by a *known* number. The $3x_1$ part is perfectly fine; one bad term is enough to break it.
- A4.** AB is 4×5 — the inner 2s match and cancel, the outer numbers survive. BA does not exist: it would need B 's 5 columns to meet A 's 4 rows, and $5 \neq 4$.
- A5.** $\det A = 3(4) - 1(2) = 10$, which is non-zero, so the inverse exists. Swap the diagonal, negate the off-diagonal, divide by 10:

$$A^{-1} = \frac{1}{10} \begin{bmatrix} 4 & -1 \\ -2 & 3 \end{bmatrix} = \begin{bmatrix} 0.4 & -0.1 \\ -0.2 & 0.3 \end{bmatrix}.$$

Check: $3(0.4) + 1(-0.2) = 1$; $3(-0.1) + 1(0.3) = 0$; $2(0.4) + 4(-0.2) = 0$; $2(-0.1) + 4(0.3) = 1$. That is I . ✓

- A6.** No. Three equations can produce at most three pivots, so at least two of the five columns must be free. If any solution exists, those two free choices generate infinitely many. So the count is 0 or infinity — never 1.
- A7.** None. Halve the second clue and it says $x + y = 5.5$, while the first says $x + y = 5$. The same quantity cannot be both, so the system is inconsistent.
- A8.** By rows: row 1 gives $1(4) + 2(1) = 6$; row 2 gives $0(4) + 3(1) = 3$. So $A\mathbf{x} = (6, 3)$.
By columns: $4 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + 1 \begin{bmatrix} 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 4 \\ 0 \end{bmatrix} + \begin{bmatrix} 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 6 \\ 3 \end{bmatrix}$. Same answer, as it must be.
- A9.** Add any multiple of $\mathbf{s}$ to $\mathbf{x}_p$. Taking $\lambda = 1$ gives $(1, 3, -1)$; taking $\lambda = -2$ gives $(1, 0, 2)$. Both work because $A(\mathbf{x}_p + \lambda\mathbf{s}) = \mathbf{b} + \lambda\mathbf{0} = \mathbf{b}$. In fact $(1, 2 + \lambda, -\lambda)$ is a solution for every real λ .
- A10.** Let p be the price of a pen and n of a notebook: $3p + 2n = 130$ and $p + 4n = 160$.
From the second, $p = 160 - 4n$. Substitute: $3(160 - 4n) + 2n = 130$, so $480 - 12n + 2n = 130$, so $-10n = -350$ and $n = 35$. Then $p = 160 - 140 = 20$.
Check: $3(20) + 2(35) = 60 + 70 = 130$ ✓ and $20 + 4(35) = 20 + 140 = 160$ ✓. A pen costs Rs. 20 and a notebook Rs. 35.

• If you remember only one page

A vector is a list; a matrix is a grid; your data is already a matrix. A linear equation is a clue in which unknowns are only scaled and added. A set of clues has 0, 1 or infinitely many answers. Pack the clues into $A\mathbf{x} = \mathbf{b}$, sweep $[A \mid \mathbf{b}]$ into a staircase, and read the answer off — particular solution plus every mix of silent moves. Then check it in the original equations.

Companion reader for Session 1 of ZC416, *Mathematical Foundations for Machine Learning and Data Science*. Prepared for the Work Integrated Learning Programmes, BITS Pilani. Read alongside the lecture slides.